

GitHub 활용 취업용 포트폴리오 완성 과정

학생 배포 자료 : 실습 매뉴얼

매뉴얼의 목표 및 구성

GitHub 를 코드 저장소가 아니라 채용 면접에서 바로 꺼내 보일 수 있는 기술 증거 체계로 바꾸는 것이 목표.
수업 후 이 자료만 다시 읽어도 README, 배포, 협업 기록, 트러블슈팅, 30 초 소개까지 스스로 개선할 수 있도록 구성

학습 축	완성 기준	면접에서 드러나는 능력
첫인상	5 초 안에 목적·역할·결과가 보임	문제 정의와 전달력
재현성	문서만 보고 실행 가능	개발 기본기와 배려
기술 깊이	대안·선택 이유·포기한 점이 있음	공학적 판단력
협업성	Issue/PR/커밋에 맥락이 남음	실무 커뮤니케이션
면접 연결	GitHub 링크로 답변을 뒷받침	경험의 구조화

권장 사용법

- 수업 중: 각 페이지의 자기 점검 질문을 본인 저장소 기준으로 표시
- 수업 후: 앞 50 페이지 기본 흐름과 뒤 30 페이지 보강 파트를 함께 따라가며 README, 데이터/보안 증거 링크, STAR 를 실제로 수정
- 면접 전: 대표 링크 3 개, 30 초 프로젝트 소개, 데이터/보안 근거 질문 5 개를 마지막으로 점검

2 일 10 시간 수업 로드맵

로드맵

1 일차는 GitHub 첫인상과 기술 설명의 골격을 만들고, 2 일차는 작동성·협업 기록·면접 답변으로 연결한다.

구간	강의 시나리오 핵심	학생 산출물
1 일차 도입	5 초 필터, 좋은/나쁜 저장소, 학습 목표	저장소 오디팅 결과
1 일차 전반	Clean Git, .gitignore, 커밋 컨벤션	정리된 저장소와 커밋 기준
1 일차 중반	README 골격, 직무별 보장	직무 맞춤 README 초안
1 일차 후반	Trade-off, 배지, Mermaid	기술 의사결정 표와 구조도
2 일차 전반	배포, CI, 실행 가이드	배포 URL 또는 재현 가능한 실행 문서
2 일차 중반	Issue, PR, 협업 기록	작업 맥락이 보이는 Issue/PR
2 일차 후반	STAR, 면접 질문, 30 초 소개	면접 답변으로 연결되는 GitHub 증거

고급 포트폴리오의 핵심 원칙

- 코드의 양보다 '의사결정의 흔적'이 먼저다.
단순한 구현을 넘어, 무엇을 해결하고자 했으며 어떤 기술적 근거(Rationale)로 해당 방식을 선택했는지 그 치열한 고민의 과정이 명확히 담겨야 한다.
- '실행 가능한 문서(Executable Documentation)'는 곧 실력이다.
데이터의 출처, 보안 검증 기준, 그리고 실행 환경이 완벽하게 재현되지 않는 포트폴리오는 면접관의 신뢰를 얻을 수 없다.
- 지원 직무에 맞춰 문서의 '서사(Narrative)'를 설계하라.
개발 직무는 '구현 과정과 기술적 선택', 데이터 직무는 '핵심 질문과 도출된 인사이트', 보안 직무는 '잠재적 위협과 방어 증거'가 스토리의 중심이 되어야 한다.
- GitHub 문서와 면접 답변은 완벽하게 동기화되어야 한다.
README의 모든 섹션은 면접관의 예상 질문과 직결되는 '단일 진실 공급원(Single Source of Truth)'으로 기능해야 한다.

체크	첫날 시작 전 확인	메모
☐	GitHub 계정과 대표 프로젝트 저장소 준비한다.	
☐	수업 중 수정할 README 파일을 생성한다.	
☐	지원 직무 또는 관심 직무를 하나 선택한다.	
☐	각 페이지 상단의 시나리오 연번으로 원본 강의 흐름과 연결해 본다.	

수업 시작 전 환경 점검

이 페이지의 결론

도구가 준비되어야 실습이 사고로 이어진다. 환경 점검은 단순 준비가 아니라 재현 가능한 개발 습관의 시작이다.

테크 리더의 렌즈

- GitHub 로그인, Git 설치, 에디터 실행, 대표 저장소 확인을 수업 시작 전에 끝낸다.
- 학생들이 자주 막히는 지점은 권한, 경로, 환경변수, 실행 명령어이다.
- 준비 상태를 문서화하면 나중에 Quick Start 작성이 쉬워진다.

작성 프레임

항목	작성 기준
준비물	GitHub 계정, Git, VS Code 또는 Cursor, Eclipse, 대표 프로젝트 저장소
검증	git --version, 저장소 clone, 로컬 실행 여부 확인
기록	막힌 지점은 실행 가이드 보완 후보로 표시

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
설치했다.	git --version 결과와 사용한 실행 명령을 README 에 기록한다.
나중에 정리하겠다.	수업 중 막힌 지점을 바로 체크해 실행 가이드 개선 항목으로 둔다.
잘 만들었다.	어떤 문제를 어떤 기준으로 해결했고 어떻게 검증했는지까지 작성한다.
나중에 설명 가능	문서만 보고도 판단할 수 있게 근거 링크와 확인 방법을 기록한다.

본인 저장소에 즉시 적용하고 README 또는 docs 폴더에 반영한다.

체크	자기 점검 질문	메모
☐	GitHub 로그인과 저장소 접근이 되는가?	
☐	로컬에서 프로젝트를 열고 실행할 수 있는가?	
☐	막힌 지점을 메모했는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

5 초 필터로 첫인상 진단

이 페이지의 결론

면접관은 처음 5 초에 읽을지 말지를 결정한다. 첫 화면은 설명이 아니라 선별 대상이다.

테크 리더의 렌즈

- README 상단은 프로젝트의 표지이자 면접관의 탐색 지도이다. 개발·데이터·보안 중 어떤 강점을 볼지 첫 화면에서 알려야 한다.
- 5 초 안에 목적, 역할, 결과, 대표 증거 링크 중 최소 3 개가 보여야 한다.
- 첫인상이 약하면 좋은 코드, 좋은 분석, 좋은 보안 검증도 확인하지 않는다.

작성 프레임

항목	작성 기준
보여야 할 것	프로젝트 한 문장 소개, 본인 역할, 핵심 기술, 대표 증거 링크
숨겨도 되는 것	긴 개발 배경, 세부 구현 설명, 전체 폴더 설명
판단 기준	처음 보는 친구가 5 초 안에 문제, 결과, 대표 증거를 말할 수 있는가?

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
React 로 만든 프로젝트	스터디 일정 조율 문제를 해결했고, 사용자 흐름과 상태관리 선택 근거를 README 에 연결했다.
기능이 많다.	핵심 기능 3 개와 각 기능이 해결하는 문제, 확인 가능한 데모 링크를 함께 제공한다.
프로젝트 설명	3 초 안에 읽히는 한 문장: 누구의 어떤 문제를 어떤 근거로 해결했는지 작성한다.
나중에 아래에 설명	상단에서 데모, 실행, Data Overview, Threat Model, STAR 링크로 바로 이동하게 만든다.

수업 중 적용

본인 저장소에 즉시 적용하고 README 또는 docs 폴더에 반영한다.

체크	자기 점검 질문	메모
☐	README 상단에서 문제, 대상, 대표 증거가 보이는가?	
☐	내 역할, 핵심 기술, 분석/검증 근거가 보이는가?	
☐	데모, 실행, 데이터/보안 증거 링크가 상단에 있는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

좋은 저장소와 나쁜 저장소

이 페이지의 결론

좋은 저장소는 면접관의 시간을 아껴 준다. 나쁜 저장소는 평가자가 추측하게 만든다.

테크 리더의 렌즈

- 평가자는 코드보다 먼저 문서 흐름, 실행성, 증거 링크, 커밋 맥락을 본다.
- 좋은 포트폴리오는 '만들었다'보다 '이 문제를 이렇게 판단했고 이렇게 검증했다'를 보여준다.
- 저장소는 코드 창고가 아니라 문제 정의, 분석 과정, 검증 결과가 묶인 기술 증거 패키지.

작성 프레임

항목	작성 기준
나쁜 신호	빈 README, 의미 없는 커밋, 실행 불가, 민감정보 노출
좋은 신호	문제 정의, 재현 가능한 실행, 구조도, 의사결정 기록
고급 신호	트러블슈팅과 PR 이 README 의 설명과 연결됨

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
기술 스택: Spring Boot, MySQL	Spring Boot 와 MySQL 을 선택한 이유, 대안, 트랜잭션 경계를 설명한다.
열심히 만들었다.	배포 오류를 해결하며 환경변수 검증 절차를 문서화했다.
잘 만들었다.	어떤 문제를 어떤 기준으로 해결했고 어떻게 검증했는지까지 작성한다.
나중에 설명 가능	문서만 보고도 판단할 수 있게 근거 링크와 확인 방법을 기록한다.

수업 중 적용

본인 저장소에 즉시 적용하고 README 또는 docs 폴더에 반영한다.

체크	자기 점검 질문	메모
☐	내 저장소의 나쁜 신호를 2 개 이상 발견했는가?	
☐	좋은 신호로 바꿀 구체적 작업을 정했는가?	
☐	고급 신호 하나를 추가할 계획이 있는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

학습 목표를 산출물로 바꾸기

이 페이지의 결론

수업 목표는 문장이 아니라 저장소에 남는 파일과 링크로 바뀌어야 한다.

테크 리더의 렌즈

- 오늘의 목표는 README, 직무별 섹션, 기술 의사결정, Mermaid 구조도에 데이터 질문과 보안 위협 흐름을 함께 연결하는 것이다.
- 내일의 목표는 배포/실행, Issue/PR, STAR, 30 초 소개이다.
- 각 산출물은 면접에서 보여줄 수 있는 증거가 되어야 하며, 데이터는 인사이트로, 보안은 검증 로그로 이어져야 한다.

작성 프레임

항목	작성 기준
목표	저장소 진단, 직무별 README, Mermaid 와 협업 기록
산출물	README.md, docs 폴더, Issue/PR 링크, 배포 URL
완성 기준	문서만 보고 프로젝트의 목적·실행·판단을 이해 가능

목표

면접관이 질문할 수 있는 판단 근거를 README 에 배치한다.

요청 흐름 또는 데이터 흐름을 Mermaid 로 설명한다.

수업 중 적용

본인 저장소에 즉시 적용하고 README 또는 docs 폴더에 반영한다.

체크	자기 점검 질문	메모
☐	오늘 남길 산출물을 파일명으로 적었는가?	
☐	내일 이어질 항목이 표시되어 있는가?	
☐	면접에서 보여줄 링크 후보를 정했는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

직무 방향 선택

이 페이지의 결론

모든 역량을 다 보여주려 하면 아무 역량도 선명하지 보여줄 수 없다. 포트폴리오는 초점을 가져야 한다.

테크 리더의 렌즈

- Frontend 는 사용자 흐름, Backend 는 구조와 안정성, Data 는 질문-데이터-인사이트-제안, Security 는 위협-검증-증거-완화를 강조한다.
- 지원 직무가 명확할수록 README 의 목차, 대표 링크, 예시 문장도 달라진다.
- 같은 프로젝트라도 직무별로 면접 질문이 다르므로, 먼저 어떤 질문을 받고 싶은지 설계한다.

작성 프레임

항목	작성 기준
Frontend	화면 흐름, 컴포넌트 구조, 상태관리, 접근성
Backend	API 설계, DB 모델링, 인증/인가, 트랜잭션
Security/Data	위협 시나리오와 통제 / 데이터 출처와 재현성

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
저는 풀스택이다.	이번 포트폴리오에서는 Backend 역량을 중심으로 API 설계와 인증 흐름을 강조한다.
여러 기술을 썼다.	지원 직무와 연결되는 2 개 기술만 깊게 설명한다.
잘 만들었다.	어떤 문제를 어떤 기준으로 해결했고 어떻게 검증했는지까지 작성한다.
나중에 설명 가능	문서만 보고도 판단할 수 있게 근거 링크와 확인 방법을 기록한다.

수업 중 적용

본인 저장소에 즉시 적용하고 README 또는 docs 폴더에 반영한다.

체크	자기 점검 질문	메모
☐	지원 직무를 하나 선택했는가?	
☐	그 직무에서 물어볼 질문 3 개를 적었는가?	
☐	내 프로젝트의 증거 위치를 찾았는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

GitHub 오디팅 체크리스트

이 페이지의 결론

오디팅은 평가가 아니라 개선 순서를 정하는 일이다. 먼저 고칠 것을 알아야 좋은 문서가 된다.

테크 리더의 렌즈

- README, 실행성, 직무 증거, 협업 기록, 데이터/보안 근거 다섯 범주로 나눈다.
- 가장 먼저 고칠 항목은 첫인상, 실행성, 대표 증거 링크.
- 기술 깊이는 구조도, 의사결정 기록, 분석/검증 근거로 보강.

작성 프레임

항목	작성 기준
README	상단 10 줄, 목차, 문제 정의, 데모 링크
실행성	Quick Start, 환경변수 예시, 테스트 방법
증거	Mermaid, Trade-off, Issue/PR, STAR

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
나중에 정리	30 분 안에 수정할 항목과 수업 후 개선할 항목을 분리한다.
전체적으로 부족	README 상단, 실행 가이드, 직무 섹션 순서로 우선순위를 잡는다.
잘 만들었다.	어떤 문제를 어떤 기준으로 해결했고 어떻게 검증했는지까지 작성한다.
나중에 설명 가능	문서만 보고도 판단할 수 있게 근거 링크와 확인 방법을 기록한다.

수업 중 적용

본인 저장소에 즉시 적용하고 README 또는 docs 폴더에 반영한다.

체크	자기 점검 질문	메모
☐	가장 먼저 고칠 항목이 하나로 정해졌는가?	
☐	오늘 수정할 항목과 수업 후 항목을 나눴는가?	
☐	면접 영향도가 높은 항목을 우선했는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

오디팅 공유와 피드백

이 페이지의 결론

피드백은 지적이 아니라 독자의 시선을 빌리는 과정이다. 내 문서의 빈틈은 남이 더 빨리 본다.

테크 리더의 렌즈

- 친구가 내 저장소를 1 분 안에 설명할 수 있으면 구조가 좋은 편이다.
- 피드백은 README, 실행성, 직무 증거, 협업 기록으로 분류한다.
- 방어하지 말고 독자가 헛갈린 지점을 기록한다.

작성 프레임

항목	작성 기준
질문	무엇을 만든 프로젝트인지 1 문장으로 말한다.
관찰	어디에서 이해가 막혔는가?
개선	상단 10 줄, 구조도, 실행 가이드 중 무엇부터 고칠 것인가?

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
친구가 잘 몰라서 그렇다.	독자가 헛갈린 지점을 README 구조 개선 신호로 본다.
설명하면 이해한다.	설명 없이 문서만 보고 이해되도록 바꾼다.
잘 만들었다.	어떤 문제를 어떤 기준으로 해결했고 어떻게 검증했는지까지 작성한다.
나중에 설명 가능	문서만 보고도 판단할 수 있게 근거 링크와 확인 방법을 기록한다.

수업 중 적용

본인 저장소에 즉시 적용하고 README 또는 docs 폴더에 반영한다.

체크	자기 점검 질문	메모
☐	동료가 내 프로젝트를 1 분 안에 설명했는가?	
☐	헛갈린 지점을 문서 수정 항목으로 바꿨는가?	
☐	피드백을 범주별로 정리했는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

Clean Git: 저장소 위생

이 페이지의 결론

저장소 위생은 실력 이전의 신뢰이다. 불필요한 파일과 민감정보는 좋은 인상을 즉시 무너뜨린다.

테크 리더의 렌즈

- node_modules, build 산출물, OS 파일, IDE 파일을 정리한다.
- **민감정보, 원본 개인정보 데이터, 실제 인증 토큰은 절대 커밋하지 않는다.**
- 실행에 필요한 예시 파일, 샘플 데이터, 안전한 테스트 계정 설명은 기록한다.

작성 프레임

항목	작성 기준
제외	node_modules, dist/build, .DS_Store, .idea, .env
보존	.env.example, README, 실행 스크립트, 샘플 데이터
주의	이미 노출된 키는 삭제가 아니라 반듯이 폐기해야한다.

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
파일이 많아도 괜찮다.	실행과 평가에 필요한 파일만 남겨 저장소의 의도를 분명히 한다.
키는 숨겨두었다.	실제 키는 저장소에서 제거하고 .env.example 에는 예시만 둔다.
잘 만들었다.	어떤 문제를 어떤 기준으로 해결했고 어떻게 검증했는지까지 작성한다.
나중에 설명 가능	문서만 보고도 판단할 수 있게 근거 링크와 확인 방법을 기록한다.

수업 중 적용

본인 저장소에 즉시 적용하고 README 또는 docs 폴더에 반영한다.

체크	자기 점검 질문	메모
☐	불필요한 파일이 커밋되어 있지 않은가?	
☐	민감정보가 없는가?	
☐	예시 설정 파일은 있는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

프로젝트별 .gitignore

이 페이지의 결론

.gitignore 는 복사 붙여넣기 파일이 아니라 프로젝트의 실행 경계를 정의하는 문서이다.

테크 리더의 렌즈

- Node, Java, Python, Data, Security 프로젝트마다 제외해야 할 파일과 공개 가능한 파일이 다르다.
- 데이터 분석 프로젝트는 원본 데이터, 개인정보, 라이선스 범위를 특히 조심한다.
- 템플릿을 적용한 뒤 노트북 출력, 캡처 로그, 샘플 데이터처럼 내 프로젝트 특수 파일 기준을 추가한다.

작성 프레임

항목	작성 기준
Node	node_modules, .env, dist, coverage
Java	target, build, .gradle, .classpath 일부
Python/Data	__pycache__, .venv, 대용량 원본 데이터, notebook checkpoint

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
GitHub 템플릿 그대로 사용	템플릿을 기반으로 내 프로젝트 생성물과 민감 파일을 추가 점검한다.
데이터도 같이 올림	샘플 데이터와 출처 설명을 두고 원본 대용량/민감 데이터는 제외한다.
잘 만들었다.	어떤 문제를 어떤 기준으로 해결했고 어떻게 검증했는지까지 작성한다.
나중에 설명 가능	문서만 보고도 판단할 수 있게 근거 링크와 확인 방법을 기록한다.

수업 중 적용

본인 저장소에 즉시 적용하고 README 또는 docs 폴더에 반영한다.

체크	자기 점검 질문	메모
☐	내 기술 스택에 맞는 .gitignore 인가?	
☐	실행에 필요한 샘플은 보존했는가?	
☐	대용량/민감 데이터가 제외되었는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

커밋 컨벤션

이 페이지의 결론

커밋 메시지는 작은 면접 기록이다. 문제를 어떻게 쪼개고 설명하는지 보여준다.

테크 리더의 렌즈

- feat, fix, docs, refactor, chore 등 Conventional Commits 를 사용한다.
- 커밋 하나에는 하나의 의도를 담는다.
- 문서 개선, 데이터 품질 보강, Threat Model 추가도 docs 커밋으로 남기면 좋은 성장 기록이 된다.

작성 프레임

항목	작성 기준
feat	새 기능 또는 사용자 가치 추가
fix	오류 수정과 재현 조건
docs	README, 실행 가이드, 구조도 개선
refactor	기능 변화 없는 구조 개선

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
update	docs: add quick start and env example
fix bug	fix: handle empty search result state
README 수정	docs: document backend request flow with Mermaid
잘 만들었다.	어떤 문제를 어떤 기준으로 해결했고 어떻게 검증했는지까지 작성한다.
나중에 설명 가능	문서만 보고도 판단할 수 있게 근거 링크와 확인 방법을 기록한다.

수업 중 적용

본인 저장소에 즉시 적용하고 README 또는 docs 폴더에 반영한다.

체크	자기 점검 질문	메모
☐	최근 커밋 5 개가 읽히는가?	
☐	문서 개선 커밋을 남겼는가?	
☐	앞으로의 커밋 규칙을 정했는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

README 상단 10 줄

이 페이지의 결론

README 상단은 면접관을 붙잡는 공간이다. 기술 목록보다 문제와 역할이 먼저 보여야 한다.

테크 리더의 렌즈

- 한 문장 소개, 본인 역할, 핵심 기술, 대표 증거 링크, 데이터/보안 강조 포인트를 배치한다.
- 상단에서 모든 것을 설명하지 말고 개발 흐름, 데이터 흐름, 보안 검증 섹션으로 안내한다.
- 장식보다 판단에 필요한 질문, 근거, 결과, 링크를 우선한다.

작성 프레임

항목	작성 기준
문장 소개	누구의 어떤 문제를 해결하는가
역할	본인이 담당한 범위와 책임
링크	배포, 데모, 실행 가이드, 주요 문서
강조	직무별 핵심 판단 1개

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
Todo 앱이다.	개인 일정 누락을 줄이기 위한 우선순위 기반 Todo 관리 앱이다.
React 사용	React Query 로 서버 상태와 캐시 무효화를 관리했다.
잘 만들었다.	어떤 문제를 어떤 기준으로 해결했고 어떻게 검증했는지까지 작성한다.
나중에 설명 가능	문서만 보고도 판단할 수 있게 근거 링크와 확인 방법을 기록한다.

수업 중 적용

본인 저장소에 즉시 적용하고 README 또는 docs 폴더에 반영한다.

체크	자기 점검 질문	메모
☐	상단 10 줄만 보고 목적이 보이는가?	
☐	본인 역할이 명확한가?	
☐	데모 또는 실행 링크가 바로 보이는가?	
☐	문제 정의, 본인 역할, 핵심 결과가 한 화면에 들어오는가?	
☐	배포 링크와 실행 가이드 링크가 모두 살아 있는가?	
☐	지원 직무와 연결되는 키워드가 자연스럽게 보이는가?	

README 전체 구조

이 페이지의 결론

좋은 README 는 길이가 아니라 읽는 순서가 좋다. 면접관이 원하는 순서로 정보를 배치해야 한다.

테크 리더의 렌즈

- 개요, 핵심 질문, 기능/분석/위험 범위, 기술 스택, 실행 방법, 증거 링크, 의사결정, 트러블슈팅, 협업 기록 순서를 권장한다.
- 모든 프로젝트가 같은 목차일 필요는 없다. 데이터형은 Business Question, 보안형은 Threat Model 이 앞에 와도 좋다.
- 지원 직무와 연결되는 섹션을 더 강하게 만들고, 나머지는 근거를 보조하는 역할로 둡니다.

작성 프레임

항목	작성 기준
앞부분	문제 정의, 역할, 결과, 데모
중간	실행 방법, 기능, 구조도
후반	기술 의사결정, 트러블슈팅, 협업 기록
부록	추가 이미지, API 문서, 참고 자료

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
목차만 많음	지원 직무와 연결되는 핵심 섹션을 먼저 배치한다.
기능 설명만 있음	기능이 왜 필요했고 어떻게 설계했는지 덧붙인다.
기술을 많이 썼다.	핵심 기술 2~3 개만 고르고 왜 사용했는지 아래 섹션으로 연결한다.
배포 링크 있음	배포 URL 옆에 테스트 계정, 주요 흐름, 실행 대안을 함께 제공한다.

수업 중 적용

본인 저장소에 즉시 적용하고 README 또는 docs 폴더에 반영한다.

체크	자기 점검 질문	메모
☐	읽는 순서가 자연스러운가?	
☐	실행 방법이 너무 아래에 묻혀 있지 않은가?	
☐	기술 판단 섹션이 있는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

Markdown 표현력

이 페이지의 결론

Markdown 은 문법 과시가 아니라 독자의 눈길을 안내하는 도구이다.

테크 리더의 렌즈

- 제목 계층을 지키고, 긴 문단은 질문·근거·판단·결과 단위로 짧게 나눕니다.
- 명령어는 코드 블록, 비교는 표, 절차는 번호 목록이 적합하다.
- 이미지와 GIF 는 핵심 흐름만 보여주며, 데이터 차트와 보안 로그에는 판단 캡션을 붙인다.

작성 프레임

항목	작성 기준
제목	H1 은 프로젝트명, H2 는 주요 섹션, H3 는 하위 설명
코드	복사 가능한 명령어와 설정 예시
표	Trade-off, API 요약, 체크리스트
이미지	사용 흐름과 결과 화면

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
문단으로 길게 설명	표나 단계로 나눠 스캔 가능하게 만든다.
이미지 여러 장 나열	대표 흐름 1~2 개와 설명을 붙인다.
문서가 깁니다.	요약, 실행, 구조, 의사결정, 트러블슈팅 순서로 탐색 경로를 만든다.
기능 설명	각 기능이 해결하는 사용자 문제와 구현 판단을 함께 작성한다.

수업 중 적용

본인 저장소에 즉시 적용하고 README 또는 docs 폴더에 반영한다.

체크	자기 점검 질문	메모
☐	제목 계층이 정리되어 있는가?	
☐	명령어가 코드 블록인가?	
☐	표와 이미지가 필요한 곳에만 쓰였는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

Frontend 포트폴리오 보강

이 페이지의 결론

Frontend 는 화면을 만들었다보다 사용자 흐름과 상태 변화를 설계했다는 증거가 중요하다.

테크 리더의 렌즈

- 화면 흐름, 컴포넌트 분리, 상태관리, 접근성, 반응형, 성능을 기록한다.
- 상태관리 선택 이유와 렌더링 최적화 근거가 있으면 강하다.
- GIF 는 핵심 사용자 흐름을 짧게 보여줘야 한다.

작성 프레임

항목	작성 기준
화면 흐름	사용자가 어떤 목표를 어떤 단계로 달성하는가
컴포넌트	재사용 기준과 책임 분리
상태관리	로컬/서버/전역 상태의 분리 이유
품질	접근성, 반응형, 로딩/에러 상태

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
React 로 구현	서버 상태는 React Query, UI 상태는 로컬 state 로 분리했다.
반응형 지원	모바일 360px 기준에서 카드 폭과 버튼 터치 영역을 검증했다.
잘 만들었다.	어떤 문제를 어떤 기준으로 해결했고 어떻게 검증했는지까지 작성한다.
나중에 설명 가능	문서만 보고도 판단할 수 있게 근거 링크와 확인 방법을 기록한다.

수업 중 적용

본인 저장소에 즉시 적용하고 README 또는 docs 폴더에 반영한다.

체크	자기 점검 질문	메모
☐	상태관리 선택 이유가 있는가?	
☐	화면 흐름이 그림이나 GIF 로 보이는가?	
☐	접근성/반응형/성능 중 하나를 설명했는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

Backend 포트폴리오 보강

이 페이지의 결론

Backend 는 요청이 들어와 응답되기까지의 구조와 예외 상황을 다루는 능력이 보여야 한다.

테크 리더의 렌즈

- API 설계, DB 모델링, 인증/인가, 예외 처리, 트랜잭션을 기록한다.
- 엔드포인트 목록보다 설계 이유가 중요하다.
- 요청 흐름 다이어그램은 면접 질문을 자연스럽게 만든다.

작성 프레임

항목	작성 기준
API	리소스 설계, 요청/응답 예시, 상태 코드
DB	엔티티 관계, 인덱스, 정규화/비정규화 판단
인증	로그인 흐름, 토큰 처리, 권한 분리
안정성	예외 처리, 트랜잭션, 로그

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
API 만들었다.	주문 생성은 재고 차감과 결제를 하나의 트랜잭션 경계로 묶었다.
DB 사용	조회 빈도가 높은 사용자-주문 관계에 인덱스를 추가했다.
UI 만들었다.	화면 흐름, 상태 분리, 로딩/에러 상태 처리까지 설명한다.
컴포넌트 분리	재사용 기준과 책임 범위를 예시 컴포넌트로 보여준다.

수업 중 적용

본인 저장소에 즉시 적용하고 README 또는 docs 폴더에 반영한다.

체크	자기 점검 질문	메모
☐	요청 흐름을 설명할 수 있는가?	
☐	DB 모델링 이유가 있는가?	
☐	예외와 권한 처리가 문서화되어 있는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

Security 포트폴리오 보강

이 페이지의 결론
Security 는 위험을 발견하고 줄이는 사고 과정이 핵심이다. '보안 적용'이라는 말만으로는 부족하다.

테크 리더의 렌즈

- Executive Summary, Threat Model, Process, Evidence, Mitigation, Lesson Learned 순서로 기록한다.
- 어떤 위험을 가정했고, 어떤 도구와 로그로 검증했으며, 완화 후 무엇이 달라졌는지 함께 작성한다.
- 실제 키와 개인정보는 문서 예시에도 넣지 않고, 캡처 자료는 반드시 마스킹한다.

작성 프레임

항목	작성 기준
위협	토큰 탈취, 권한 상승, 민감정보 노출
통제	환경변수, RBAC, 입력 검증, 로그 마스킹
검증	권한별 접근 테스트, 실패 케이스, 보안 체크리스트
문서	보안 고려사항과 남은 한계

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
JWT 사용	Access Token 만료 시간을 짧게 두고 Refresh Token 저장 위치를 분리했다.
권한 처리	관리자 API 는 역할 기반 접근 제어와 실패 응답을 테스트했다.
API 구현	요청 흐름, 상태 코드, 예외 처리, 트랜잭션 경계를 함께 문서화한다.
DB 연결	조회 패턴과 데이터 일관성을 기준으로 모델링 이유를 설명한다.

수업 중 적용

본인 저장소에 즉시 적용하고 README 또는 docs 폴더에 반영한다.

체크	자기 점검 질문	메모
☐	위협과 대응이 짝지어져 있는가?	
☐	검증 방법이 있는가?	
☐	민감정보가 문서에 노출되지 않았는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

Data 포트폴리오 보강

이 페이지의 결론

Data 는 그래프보다 데이터가 어떤 기준으로 만들어졌고 다시 재현되는지가 중요하다.

테크 리더의 렌즈

- Business Question, Data Overview, Methodology, Key Insights, Actionable Proposal 순서로 기록한다.
- 분석 결과, 한계, 다음 검증 계획을 함께 쓰면 신뢰가 높아집니다.
- 노트북만 올리지 말고 데이터 출처, 실행 순서, 환경, 차트가 답하는 질문을 설명한다.

작성 프레임

항목	작성 기준
출처	데이터 제공처, 수집 기간, 라이선스
정제	결측치, 이상치, 중복 제거 기준
분석	지표 정의, 모델/통계 방법, 검증 기준
재현	환경, 실행 순서, 결과 파일

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
데이터 분석	2024 년 공공데이터를 기준으로 결측치 3%를 제거하고 월별 평균을 비교했다.
정확도 높음	검증셋 기준 F1-score 0.82 이며 클래스 불균형 한계를 남겼다.
보안 검증	예상 위협, 적용한 통제, 검증 방법을 한 세트로 작성한다.
권한 처리	역할별 접근 가능/불가능 시나리오와 실패 응답을 기록한다.

수업 중 적용

본인 저장소에 즉시 적용하고 README 또는 docs 폴더에 반영한다.

체크	자기 점검 질문	메모
☐	데이터 출처가 명확한가?	
☐	전처리 기준이 재현 가능한가?	
☐	한계와 개선 방향이 있는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

직무별 섹션 작성

이 페이지의 결론

직무별 섹션은 기술명 목록이 아니라 면접 질문에 답할 근거이다.

테크 리더의 렌즈

- 섹션 제목은 아키텍처, 기술적 의사결정, 보안 고려사항, 데이터 흐름처럼 구체적으로 작성한다.
- 문제, 질문, 선택, 검증, 결과 순서로 작성한다.
- 지원 직무와 무관한 장황한 설명은 줄인다.

작성 프레임

항목	작성 기준
문제	어떤 요구사항 또는 제약이 있었는가
선택	어떤 구조나 기술을 선택했는가
이유	대안과 비교해 왜 적합했는가
결과	무엇이 개선되거나 가능해졌는가

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
기술적 의사결정	검색 지연을 줄이기 위해 서버 페이지네이션과 클라이언트 캐싱을 함께 적용했다.
아키텍처	인증, 주문, 결제 책임을 모듈로 분리해 변경 영향을 줄였다.
분석했다.	데이터 출처, 전처리 기준, 지표 정의, 재현 명령을 함께 제공한다.
결과가 좋다.	수치, 검증 기준, 한계, 다음 개선 방향까지 작성한다.

수업 중 적용

본인 저장소에 즉시 적용하고 README 또는 docs 폴더에 반영한다.

체크	자기 점검 질문	메모
☐	섹션 제목이 직무와 연결되는가?	
☐	문제-선택-이유-결과가 있는가?	
☐	면접 질문으로 확장 가능한가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

기술 의사결정

이 페이지의 결론

면접에서 중요한 것은 '무엇을 썼는가?'가 아니라 '왜 그렇게 선택했는가?' 이다.

테크 리더의 렌즈

- 기술 선택, 분석 방법 선택, 보안 통제 선택에는 항상 대안과 포기한 점이 있다.
- 제약 조건을 드러내면 판단이 현실적으로 보인다.
- 완벽한 선택보다 근거 있는 선택이 더 좋다.

작성 프레임

항목	작성 기준
선택 주제	상태관리, 인증 방식, DB 설계, 배포 방식 등
대안	비교 가능한 다른 방법
선택 이유	프로젝트 조건과 연결된 근거
포기한 점	복잡도, 비용, 성능, 학습 곡선 등

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
좋아서 선택	초기 개발 속도와 팀 숙련도를 고려해 Spring Security 기본 구성을 선택했다.
최고의 기술	규모가 작아 Kafka 대신 단순 큐와 재시도 정책으로 충분하다고 판단했다.
잘 만들었다.	어떤 문제를 어떤 기준으로 해결했고 어떻게 검증했는지까지 작성한다.
나중에 설명 가능	문서만 보고도 판단할 수 있게 근거 링크와 확인 방법을 기록한다.

수업 중 적용

본인 저장소에 즉시 적용하고 README 또는 docs 폴더에 반영한다.

체크	자기 점검 질문	메모
☐	대안이 하나 이상 있는가?	
☐	선택 이유가 프로젝트 조건과 연결되는가?	
☐	포기한 점이 솔직하게 적혔는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

Trade-off 표 고급화

이 페이지의 결론

Trade-off 표는 짧은 공간에서 공학적 판단력을 보여주는 가장 효율적인 형식이다.

테크 리더의 렌즈

- 표에는 선택한 방법, 대안, 선택 이유, 포기한 점, 검증 방법, 증거 링크를 넣는다.
- 검증 방법이 들어가면 단순 의견이 아니라 판단의 근거가 된다.
- 너무 사소한 선택은 제외한다.

작성 프레임

항목	작성 기준
주제	면접 질문이 나올 만한 중요한 선택
선택/대안	비교 가능한 후보
근거	요구사항, 팀 상황, 성능, 안정성
검증	테스트, 로그, 사용자 흐름, 수치

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
React Query 사용	서버 상태 캐싱과 재요청 제어가 필요해 React Query 를 선택했고, 전역 상태 복잡도를 줄였다.
JWT 사용	세션 서버 부담을 줄이되 토큰 만료와 재발급 흐름을 명확히 문서화했다.
좋아서 선택	프로젝트 제약, 대안 비교, 포기한 점을 기준으로 선택 이유를 작성한다.
유명한 기술	팀 숙련도, 유지보수성, 배포 환경과의 적합성을 근거로 작성한다.

수업 중 적용

본인 저장소에 즉시 적용하고 README 또는 docs 폴더에 반영한다.

체크	자기 점검 질문	메모
☐	표에 검증 방법이 있는가?	
☐	사소한 취향 선택이 아닌가?	
☐	포기한 점이 면접에서 설명 가능한가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

배지와 데모

이 페이지의 결론

배지와 데모는 장식이 아니라 신뢰 신호이다. 많을수록 좋은 것이 아니라 의미 있을수록 좋다.

테크 리더의 렌즈

- 배지는 기술 스택, 빌드 상태, 배포 상태, 테스트 상태 정도만 사용한다.
- 데모 GIF 는 가장 중요한 사용자 흐름 하나를 보여준다.
- 깨진 배지와 오래된 GIF 는 오히려 감점 요인이다.

작성 프레임

항목	작성 기준
배지	build passing, deploy status, tech stack, license
스크린샷	핵심 화면과 결과 상태
GIF	문제 해결 흐름 또는 대표 기능
주의	개인정보, 토큰, 내부 URL 노출 금지

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
배지가 많음	평가에 의미 있는 3~5 개만 기록한다.
화면 캡처만 있음	캡처 아래에 무엇을 판단하면 되는지 설명한다.
A 가 더 좋음	성능, 복잡도, 학습 비용, 운영 부담 중 어떤 기준에서 나은지 작성한다.
대안 없음	사용하지 않은 대안을 하나라도 적고 왜 제외했는지 설명한다.

수업 중 적용

본인 저장소에 즉시 적용하고 README 또는 docs 폴더에 반영한다.

체크	자기 점검 질문	메모
☐	배지가 의미 있는 정보만 담는가?	
☐	데모가 핵심 흐름을 보여주는가?	
☐	이미지에 민감정보가 없는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

Mermaid 구조도

이 페이지의 결론

Mermaid 는 GitHub 안에서 구조를 코드처럼 관리하는 도구이다. 긴 설명을 그림으로 압축한다.

테크 리더의 렌즈

- Frontend 는 화면/컴포넌트 흐름, Backend 는 API/DB 흐름, Security 는 인증/권한 흐름, Data 는 파이프라인을 그린다.
- 완벽한 그림보다 데이터 흐름, 요청 흐름, 위협 경계를 질문으로 만들 수 있는 단순한 구조가 좋다.
- 그림 아래에는 반드시 해석 문장을 붙인다.

작성 프레임

항목	작성 기준
선택	가장 설명하기 어려운 흐름 하나
표현	노드 수를 줄이고 흐름 방향을 명확히
해석	왜 이 구조가 필요한지 한 문단으로 설명
관리	README 안에서 바로 렌더링 확인

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
구조도만 있음	구조도 아래에 책임 분리와 데이터 흐름을 설명한다.
너무 복잡함	면접에서 설명할 핵심 흐름 하나만 기록한다.
잘 만들었다.	어떤 문제를 어떤 기준으로 해결했고 어떻게 검증했는지까지 작성한다.
나중에 설명 가능	문서만 보고도 판단할 수 있게 근거 링크와 확인 방법을 기록한다.

수업 중 적용

본인 저장소에 즉시 적용하고 README 또는 docs 폴더에 반영한다.

체크	자기 점검 질문	메모
☐	그림으로 설명할 흐름을 골랐는가?	
☐	용어가 과하지 않은가?	
☐	README 에서 정상 렌더링되는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

Mermaid 템플릿

이 페이지의 결론

템플릿은 복사해서 끝내는 것이 아니라 내 프로젝트의 책임과 흐름을 드러내도록 바뀌어야 한다.

테크 리더의 렌즈

- flowchart 는 흐름, sequenceDiagram 은 요청과 응답, ERD 는 데이터 관계에 적합하다.
- 직무별로 같은 템플릿도 강조점이 달라진다. 데이터는 흐름과 지표, 보안은 경계와 통제를 더 분명히 표시한다.
- 노드 이름은 면접관이 이해할 수 있는 용어로 작성한다.

작성 프레임

항목	작성 기준
Frontend	User -> Page -> Component -> State -> API
Backend	Client -> Controller -> Service -> Repository -> DB
Security	Login -> Token -> Role Check -> Protected API
Data	Raw -> Clean -> Feature -> Analyze -> Visualize

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
API 호출	검색 요청 -> Query Parameter 검증 -> Repository 조회 -> 캐시 갱신
데이터 처리	수집 -> 결측치 처리 -> 지표 계산 -> 시각화 결과 저장
잘 만들었다.	어떤 문제를 어떤 기준으로 해결했고 어떻게 검증했는지까지 작성한다.
나중에 설명 가능	문서만 보고도 판단할 수 있게 근거 링크와 확인 방법을 기록한다.

수업 중 적용

본인 저장소에 즉시 적용하고 README 또는 docs 폴더에 반영한다.

체크	자기 점검 질문	메모
☐	내 직무에 맞는 다이어그램 유형인가?	
☐	노드가 너무 많지 않은가?	
☐	그림 아래 해석 문장이 있는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

다이어그램 피어리뷰

이 페이지의 결론

좋은 다이어그램은 설명 없이도 1분 안에 구조를 읽을 수 있다

테크 리더의 렌즈

- 옆 사람이 구조를 설명하지 못하면 용어가 많거나 흐름이 불명확한 것이다.
- 피어리뷰에서는 정확성보다 이해 가능성을 먼저 본다.
- 막히는 지점은 README 문장으로 보완한다.

작성 프레임

항목	작성 기준
1분 설명	상대방이 다이어그램만 보고 흐름 설명
질문 생성	면접 질문이 될 만한 노드 표시
수정	용어 단순화, 방향 정리, 해석 문장 추가

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
다 아는 내용	처음 보는 사람 기준으로 노드와 흐름을 다시 점검한다.
그림이 복잡함	핵심 흐름과 부가 흐름을 분리한다.
잘 만들었다.	어떤 문제를 어떤 기준으로 해결했고 어떻게 검증했는지까지 작성한다.
나중에 설명 가능	문서만 보고도 판단할 수 있게 근거 링크와 확인 방법을 기록한다.

수업 중 적용

본인 저장소에 즉시 적용하고 README 또는 docs 폴더에 반영한다.

체크	자기 점검 질문	메모
☐	동료가 1분 안에 설명했는가?	
☐	이해가 막힌 용어를 줄였는가?	
☐	면접 질문 후보를 표시했는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

1 일차 최종 정리

이 페이지의 결론

1 일차의 목표는 GitHub 첫인상을 설계하고, 기술적 서사의 프레임워크를 구축하는 것이다..

테크 리더의 렌즈

- README 상단, Clean Git, 직무별 섹션, Trade-off, Mermaid 가 오늘의 핵심 산출물이다.
- 완성보다 중요한 것은 내일 이어서 보완할 기준을 명확히 세우는 것이다.
- 빠진 항목은 내일 배포, 협업 기록, 데이터/보안 증거 링크를 작성하면서 연결한다.

작성 프레임

항목	작성 기준
필수	README 상단, 실행 가이드 초안, 직무별 섹션
권장	Trade-off 표, Mermaid 구조도, 배지/데모
Link 연결	배포 URL, CI, Issue/PR, STAR, 30 초 소개

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
오늘 다 못함	빠진 항목을 Issue 로 만들고 내일 작업 단위로 연결한다.
README 만 작성	직무별 증거와 구조도를 추가해야 면접 질문으로 이어진다.
잘 만들었다.	어떤 문제를 어떤 기준으로 해결했고 어떻게 검증했는지까지 작성한다.
나중에 설명 가능	문서만 보고도 판단할 수 있게 근거 링크와 확인 방법을 기록한다.

수업 중 적용

본인 저장소에 즉시 적용하고 README 또는 docs 폴더에 반영한다.

체크	자기 점검 질문	메모
☐	README 상단이 개선되었는가?	
☐	직무별 섹션이 있는가?	
☐	Trade-off 와 Mermaid 초안이 있는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

전일 산출물 확인

이 페이지의 결론

2 일차는 문서를 실제 작동하는 포트폴리오와 면접 답변으로 연결하는 날이다.

테크 리더의 렌즈

- README 상단, 직무별 섹션, Mermaid, Trade-off 를 먼저 확인한다.
- 빠진 항목은 오늘 Issue 로 만들고 데이터 품질, 보안 검증, 문서 보강 작업 기록으로 기록한다.
- 문서와 실제 저장소 상태가 다르면 신뢰가 떨어진다.

작성 프레임

항목	작성 기준
확인	README, 직무별 섹션, 구조도, 의사결정
보완	배포, 실행 가이드, CI, Issue/PR
연결	STAR 와 30 초 소개를 README 링크와 연결

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
어제 작성했다.	실제 GitHub 에서 렌더링과 링크 동작을 확인한다.
빠진 게 있다.	빠진 항목을 오늘 작업 Issue 로 만들고 PR 에 연결한다.
잘 만들었다.	어떤 문제를 어떤 기준으로 해결했고 어떻게 검증했는지까지 작성한다.
나중에 설명 가능	문서만 보고도 판단할 수 있게 근거 링크와 확인 방법을 기록한다.

수업 중 적용

관련 링크를 README 에 연결해 면접에서 바로 보여줄 수 있게 정리한다.

체크	자기 점검 질문	메모
☐	전일 산출물이 GitHub 에 반영되었는가?	
☐	깨진 링크가 없는가?	
☐	오늘 보완할 항목이 정해졌는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

배포의 의미

이 페이지의 결론

배포 URL 은 '코드를 작성했다'를 넘어 '동작하는 결과를 제공할 수 있다'는 강한 증거이다.

테크 리더의 렌즈

- 프론트엔드는 Vercel, Netlify, GitHub Pages 가 적합할 수 있다.
- 백엔드와 데이터 프로젝트는 배포 대신 재현 가능한 실행 가이드가 더 적합할 수 있다.
- 무리한 배포보다 안정적인 확인 방법이 중요하며, 데이터 샘플과 보안 테스트 범위도 함께 안내한다.

작성 프레임

항목	작성 기준
정적 웹	Vercel, Netlify, GitHub Pages
API 서버	Render, Railway, Docker, Swagger, Postman
Data	Notebook, requirements, 실행 순서, 결과 재현
판단	비용, 보안, 유지 가능성

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
배포했다.	배포 URL, 배포 방식, 환경변수 주의사항을 README 상단에 작성한다.
배포가 어렵다.	Docker 또는 로컬 실행 가이드로 재현성을 보장한다.
잘 만들었다.	어떤 문제를 어떤 기준으로 해결했고 어떻게 검증했는지까지 작성한다.
나중에 설명 가능	문서만 보고도 판단할 수 있게 근거 링크와 확인 방법을 기록한다.

수업 중 적용

관련 링크를 README 에 연결해 면접에서 바로 보여줄 수 있게 정리한다.

체크	자기 점검 질문	메모
☐	내 프로젝트에 맞는 확인 방식인가?	
☐	배포 URL 또는 실행 대안이 있는가?	
☐	비용과 보안 문제가 없는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

Quick Start 고급 작성

이 페이지의 결론

Quick Start 는 친절한 문서가 아니라 재현 가능한 개발 능력의 증거이다.

테크 리더의 렌즈

- clone, install, env, sample data, run, test 순서로 작성한다.
- 버전, 환경변수, 샘플 데이터, 테스트 계정, 실패 시 대처를 포함하면 수준이 올라간다.
- 명령어는 실제로 복사해서 실행 가능해야 한다.

작성 프레임

항목	작성 기준
1	git clone 과 프로젝트 이동
2	의존성 설치와 버전 조건
3	.env.example 기반 환경변수 설정
4	실행, 테스트, 접속 URL
5	자주 나는 오류와 해결

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
npm install 후 실행	Node 20 기준 npm install, npm run dev 후 http://localhost:3000 접속
환경변수 설정	.env.example 을 복사하고 API_BASE_URL 값을 입력한다.
잘 만들었다.	어떤 문제를 어떤 기준으로 해결했고 어떻게 검증했는지까지 작성한다.
나중에 설명 가능	문서만 보고도 판단할 수 있게 근거 링크와 확인 방법을 기록한다.

```
## Quick Start
git clone https://github.com/user/repo.git
cd repo
npm install
cp .env.example .env
# optional: load sample-data/ for data demo
npm run dev
npm test
```

수업 중 적용

관련 링크를 README 에 연결해 면접에서 바로 보여줄 수 있게 정리한다.

체크	자기 점검 질문	메모
☐	새 폴더에서 문서만 보고 실행했는가?	
☐	환경변수 예시가 있는가?	

☐	오류 대처가 적혀 있는가?	
☐	clone 직후 문서만 보고 실행 가능한가?	
☐	환경변수 누락 시 어떤 오류가 나는지 설명했는가?	
☐	테스트 명령과 성공 기준을 적었는가?	

환경변수와 테스트 계정

이 페이지의 결론

실행 가이드에서 가장 자주 막히는 지점은 환경변수와 계정 정보 부분이다.

테크 리더의 렌즈

- .env.example 에는 키 이름과 예시값만 두고, 데이터 파일과 보안 캡처는 공개 가능 범위를 분리한다.
- 테스트 계정은 권한과 사용 범위를 명확히 적고, 보안 검증용 계정은 별도로 표시한다.
- 실제 비밀번호, 개인정보, 내부 URL, 원본 민감 데이터는 문서에 넣지 않는다.

작성 프레임

항목	작성 기준
환경변수	이름, 용도, 예시값, 필수 여부
테스트 계정	역할, 접근 범위, 초기화 주기
보안	실제 키 미포함, 만료 가능한 계정 사용

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
API_KEY=abcd	API_KEY=your-api-key
test@test.com / 1234	데모 계정은 읽기 전용이며 수업 종료 후 비활성화된다.
npm install 후 실행	버전, 환경변수, 실행 URL, 테스트 명령, 실패 시 대처를 순서대로 작성한다.
로컬에서 됨	깨끗한 새 폴더에서 clone 부터 실행해 검증한 결과를 기록한다.

수업 중 적용

관련 링크를 README 에 연결해 면접에서 바로 보여줄 수 있게 정리한다.

체크	자기 점검 질문	메모
☐	.env.example 이 있는가?	
☐	실제 비밀번호가 없는가?	
☐	테스트 계정 범위를 설명했는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

CI 배지와 자동화

이 페이지의 결론

CI 배지는 프로젝트가 자동으로 검증되는 흐름을 보여줄 수 있지만, 깨진 배지는 역효과이다.

테크 리더의 렌즈

- GitHub Actions 를 사용한다면 빌드/테스트 상태 배지를 README 에 연결한다.
- 자동화가 없다면 수동 테스트 명령, 데이터 검증 절차, 보안 점검 절차를 정확히 작성한다.
- 배지는 실제 워크플로와 연결되어야 한다.

작성 프레임

항목	작성 기준
결과가 있을 때	build, test, lint, deploy 상태 표시
결과가 없을 때	수동 실행과 검증 명령 명확히 작성
주의	실패 중인 배지, 의미 없는 배지, 오래된 상태

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
CI 적용	push 시 test workflow 가 실행되고 README 배지로 상태를 확인한다.
테스트 없음	npm test 또는 ./gradlew test 명령과 예상 결과를 문서화한다.
잘 만들었다.	어떤 문제를 어떤 기준으로 해결했고 어떻게 검증했는지까지 작성한다.
나중에 설명 가능	문서만 보고도 판단할 수 있게 근거 링크와 확인 방법을 기록한다.

수업 중 적용

관련 링크를 README 에 연결해 면접에서 바로 보여줄 수 있게 정리한다.

체크	자기 점검 질문	메모
☐	배지가 실제 워크플로와 연결되는가?	
☐	실패 상태를 방지하지 않았는가?	
☐	수동 검증 방법이 있는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

실행 가이드 검증

이 페이지의 결론

실행 가이드는 작성한 사람이 아니라 처음 보는 사람이 성공해야 완성이다.

테크 리더의 렌즈

- 새 폴더에서 clone 부터 실행하고, 샘플 데이터와 테스트 계정만으로 재현되는지 확인한다.
- 기존 캐시와 로컬 설정에 기대면 검증이 아니다.
- 막힌 단계는 문서의 품질을 높이는 단서이며, 데이터/보안 조건 누락도 함께 점검한다.

작성 프레임

항목	작성 기준
검증 환경	새 폴더, 가능하면 다른 PC 또는 친구 PC
검증 순서	clone -> install -> env -> run -> test
수정	막힌 단계마다 원인과 해결 추가

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
제 컴퓨터에서는 된다.	새 환경에서 문서만 보고 실행되는지 확인했다.
오류가 나면 원인을 기록한다.	자주 나는 오류와 해결 방법을 README 에 추가한다.
잘 만들었다.	어떤 문제를 어떤 기준으로 해결했고 어떻게 검증했는지까지 작성한다.
나중에 설명 가능	문서만 보고도 판단할 수 있게 근거 링크와 확인 방법을 기록한다.

수업 중 적용

관련 링크를 README 에 연결해 면접에서 바로 보여줄 수 있게 정리한다.

체크	자기 점검 질문	메모
☐	새 환경에서 실행했는가?	
☐	막힌 지점이 문서에 반영되었는가?	
☐	접속 URL 과 테스트 방법이 명확한가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

Issue 의 역할

이 페이지의 결론

Issue 는 할 일 목록이 아니라 문제 정의 능력의 증거이다.

테크 리더의 렌즈

- 좋은 Issue 는 배경, 할 일, 완료 기준, 확인 가능한 증거를 포함한다.
- 혼자 하는 프로젝트라도 Issue 가 있으면 작업 맥락이 남는다.
- README 개선, Data Overview 보강, Threat Model 추가도 훌륭한 Issue 주제이다.

작성 프레임

항목	작성 기준
배경	왜 이 작업이 필요한가
할 일	작업 범위를 체크박스로 정리
완료 기준	무엇이 되면 끝인가
참고	화면, 오류, 관련 링크

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
수정하기	README 상단에 배포 URL 과 Quick Start 링크를 추가한다.
버그 고치기	검색어가 없을 때 빈 상태 메시지를 보여주도록 수정한다.
잘 만들었다.	어떤 문제를 어떤 기준으로 해결했고 어떻게 검증했는지까지 작성한다.
나중에 설명 가능	문서만 보고도 판단할 수 있게 근거 링크와 확인 방법을 기록한다.

수업 중 적용

관련 링크를 README 에 연결해 면접에서 바로 보여줄 수 있게 정리한다.

체크	자기 점검 질문	메모
☐	Issue 제목이 구체적인가?	
☐	배경과 완료 기준이 있는가?	
☐	README 개선 Issue 도 남겼는가?	
☐	Issue 하나가 하나의 문제를 다루는가?	
☐	완료 기준이 체크박스로 확인 가능한가?	
☐	관련 README 섹션이나 PR 로 연결되어 있는가?	

Issue 템플릿

이 페이지의 결론

템플릿은 반복 작업의 품질을 일정하게 만드는 장치이다.

테크 리더의 렌즈

- 기능 개선, 버그 수정, 문서 보강 Issue 를 구분하면 좋다.
- 완료 기준은 반드시 확인 가능한 문장으로 쓰고, 차트·로그·테스트 링크 중 하나로 증명한다.
- 면접에서 보여줄 Issue 는 작업 맥락이 분명해야 한다.

작성 프레임

항목	작성 기준
기능	사용자 가치, 구현 범위, 완료 기준
버그	재현 절차, 기대 결과, 실제 결과
문서	읽는 사람, 부족한 정보, 보완 위치

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
문서 정리	처음 실행하는 사용자를 위해 환경변수 설명과 실행 오류 해결을 추가한다.
버그	로그인 실패 시 오류 메시지가 사라지는 문제를 재현 절차와 함께 기록한다.
할 일	배경, 작업 범위, 완료 기준, 참고 링크가 있는 Issue 로 바꾼다.
README 수정	누가 어떤 정보를 찾기 어려웠는지 문제 정의부터 적다.

수업 중 적용

관련 링크를 README 에 연결해 면접에서 바로 보여줄 수 있게 정리한다.

체크	자기 점검 질문	메모
☐	Issue 유형이 구분되는가?	
☐	완료 기준이 체크 가능한가?	
☐	관련 README 섹션이 연결되는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

Pull Request 의 역할

이 페이지의 결론

PR 은 변경한 결과보다 변경한 이유와 확인 방법이 중요하다.

테크 리더의 렌즈

- 무엇을 바꿨는지, 왜 바꿨는지, 어떤 데이터/보안 근거로 확인했는지 적다.
- 화면 변화가 있으면 스크린샷이나 GIF 를 붙인다.
- Issue 와 연결하면 작업 흐름이 살아납니다.

작성 프레임

항목	작성 기준
변경	수정 파일과 핵심 내용
이유	관련 문제, 요구사항, 개선 목표
확인	실행 명령, 테스트 결과, 화면 확인
연결	Closes #번호, 관련 README 링크

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
수정 완료	README Quick Start 를 추가하고 새 환경에서 npm run dev 까지 확인했다.
작업했다.	Issue #3 의 완료 기준 3 개를 모두 확인했다.
잘 만들었다.	어떤 문제를 어떤 기준으로 해결했고 어떻게 검증했는지까지 작성한다.
나중에 설명 가능	문서만 보고도 판단할 수 있게 근거 링크와 확인 방법을 기록한다.

수업 중 적용

관련 링크를 README 에 연결해 면접에서 바로 보여줄 수 있게 정리한다.

체크	자기 점검 질문	메모
☐	PR 설명에 변경 이유가 있는가?	
☐	확인 방법이 있는가?	
☐	Issue 가 연결되는가?	
☐	PR 설명만 읽어도 변경 이유가 보이는가?	
☐	확인 방법이 명령어, 스크린샷, 로그 중 하나로 제시되는가?	
☐	리뷰어가 보기 좋은 크기의 변경인가?	

작은 PR 만들기

이 페이지의 결론

좋은 PR 은 크기가 아니라 의도가 선명하다. 작은 변경이 오히려 실무 감각을 보여준다.

테크 리더의 렌즈

- 브랜치 이름은 작업 내용을 설명한다.
- 문서 보강, 구조도 추가, 데이터 품질 설명, 보안 검증 로그 추가도 좋은 PR 이다.
- 한 PR 에 여러 목적을 섞지 않는다.

작성 프레임

항목	작성 기준
브랜치	docs/add-quick-start, fix/search-empty-state
커밋	작업 단위별 명확한 메시지
PR	변경 이유, 확인 방법, 연결 Issue

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
big-update	docs/add-mermaid-architecture
여러 기능 수정	README 보강 PR 과 코드 수정 PR 을 분리한다.
수정 완료	변경 이유, 확인 방법, 관련 Issue, 스크린샷 또는 로그를 함께 기록한다.
작업했다.	무엇이 달라졌고 어떻게 검증했는지 리뷰어 관점으로 작성한다.

수업 중 적용

관련 링크를 README 에 연결해 면접에서 바로 보여줄 수 있게 정리한다.

체크	자기 점검 질문	메모
☐	PR 하나의 목적이 명확한가?	
☐	브랜치명이 작업을 설명하는가?	
☐	문서 개선도 PR 로 남겼는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

협업 기록을 README 에 연결

이 페이지의 결론

Issue 와 PR 이 있어도 README 에서 찾기 어렵다면 포트폴리오 증거로 약하다.

테크 리더의 렌즈

- README 에 주요 개선 기록을 선별해 링크하고, 각 링크가 개발·데이터·보안 중 무엇을 증명하는지 적다.
- 모든 Issue 를 나열하지 말고 면접 질문, 분석 질문, 보안 검증 질문과 연결되는 것을 고릅니다.
- 링크에는 무엇을 보여주는지 한 문장 설명을 붙인다.

작성 프레임

항목	작성 기준
선별 기준	기술 판단, 문제 해결, 구조 개선, 배포/실행 개선
표현	링크 + 보여주는 능력
주의	링크만 나열하지 않기

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
PR #5	PR #5: Quick Start 를 추가해 새 환경 실행 실패 지점을 해결했다.
Issue 목록	대표 Issue 3 개를 문제 정의, 문서 보강, 트러블슈팅으로 분류한다.
잘 만들었다.	어떤 문제를 어떤 기준으로 해결했고 어떻게 검증했는지까지 작성한다.
나중에 설명 가능	문서만 보고도 판단할 수 있게 근거 링크와 확인 방법을 기록한다.

수업 중 적용

관련 링크를 README 에 연결해 면접에서 바로 보여줄 수 있게 정리한다.

체크	자기 점검 질문	메모
☐	대표 Issue/PR 이 README 에서 보이는가?	
☐	링크 설명이 있는가?	
☐	너무 많은 링크를 나열하지 않았는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

STAR 트러블슈팅

이 페이지의 결론

트러블슈팅은 오류 해결 기록이 아니라 문제를 정의하고 검증한 경험의 증거이다.

테크 리더의 렌즈

- Situation 은 상황, Task 는 과제, Action 은 행동, Result 는 결과이다.
- 면접 답변으로 쓰려면 내 행동, 판단 기준, 데이터/보안 검증 결과가 분명해야 한다.
- 단순 오타 수정은 STAR 주제로 약하다.

작성 프레임

항목	작성 기준
S	언제 어떤 상황에서 문제가 발생했는가
T	내가 해결해야 한 과제와 제약은 무엇인가
A	어떤 분석과 조치를 했는가
R	무엇이 개선되었고 어떻게 확인했는가

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
오류가 났다.	Vercel 배포에서 환경변수 누락으로 API 요청이 실패했다.
고쳤다.	환경변수 검증 절차와 .env.example 을 추가해 재발을 줄였다.
잘 만들었다.	어떤 문제를 어떤 기준으로 해결했고 어떻게 검증했는지까지 작성한다.
나중에 설명 가능	문서만 보고도 판단할 수 있게 근거 링크와 확인 방법을 기록한다.

수업 중 적용

관련 링크를 README 에 연결해 면접에서 바로 보여줄 수 있게 정리한다.

체크	자기 점검 질문	메모
☐	실제 겪은 문제인가?	
☐	내 행동이 드러나는가?	
☐	결과가 확인 가능한가?	
☐	상황과 과제가 섞이지 않고 분리되어 있는가?	
☐	Action 에 원인 분석 과정이 들어 있는가?	
☐	Result 가 수치나 재현 결과로 검증되는가?	

STAR 사례 고르기

이 페이지의 결론

좋은 STAR 주제는 기술 질문으로 확장될 수 있는 문제이다.

테크 리더의 렌즈

- 배포 오류, 상태관리 문제, N+1 문제, 인증 문제, 데이터 결측치 처리, 성능 저하가 좋은 후보이다.
- 프로젝트 핵심 기능, 핵심 분석 질문, 핵심 보안 위험과 관련 있을수록 좋다.
- 원인 분석 또는 위험 검증 과정이 있어야 한다.

작성 프레임

항목	작성 기준
강한 주제	성능 저하, 인증 실패, 배포 오류, 데이터 품질 문제
약한 주제	오타, 단순 스타일 수정, 라이브러리 설치 실수
선택 기준	재현 가능, 원인 분석 가능, 결과 확인 가능

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
버그 고침	검색 API 가 빈 결과를 반환할 때 UI 가 멈춘 원인을 상태 처리 누락으로 분석했다.
배포 성공	환경변수 누락 원인을 찾아 배포 체크리스트를 추가했다.
오류 해결	상황, 과제, 행동, 결과를 분리하고 재현 절차와 검증 결과를 연결한다.
문제가 사라짐	같은 조건에서 다시 실행했을 때 실패가 재현되지 않았다는 확인을 기록한다.

수업 중 적용

관련 링크를 README 에 연결해 면접에서 바로 보여줄 수 있게 정리한다.

체크	자기 점검 질문	메모
☐	문제가 구체적인가?	
☐	원인 분석 과정이 있는가?	
☐	핵심 기능과 관련 있는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

STAR 작성: 상황과 과제

이 페이지의 결론

Situation 과 Task 는 길게 쓰는 곳이 아니라 문제의 맥락과 목표를 정확히 세우는 곳이다.

테크 리더의 렌즈

- 상황에는 발생 위치, 데이터/보안 조건, 증상, 영향 범위를 작성한다.
- 과제에는 해결 목표, 검증 기준, 제약 조건을 작성한다.
- 모호한 표현보다 관찰 가능한 사실이 좋다.

작성 프레임

항목	작성 기준
Situation	배포 후 로그인 요청이 401로 실패했고 사용자 진입이 막힘
Task	환경변수 설정 누락을 찾아 재배포 없이 재현 절차를 문서화
주의	감정 표현보다 증상과 영향

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
문제가 있었다.	배포 환경에서 API_BASE_URL 이 비어 있어 로그인 요청이 잘못된 주소로 전송되었다.
해결해야 했다.	원인을 분리하고 재발 방지 문서를 추가해야 했다.
잘 만들었다.	어떤 문제를 어떤 기준으로 해결했고 어떻게 검증했는지까지 작성한다.
나중에 설명 가능	문서만 보고도 판단할 수 있게 근거 링크와 확인 방법을 기록한다.

수업 중 적용

관련 링크를 README 에 연결해 면접에서 바로 보여줄 수 있게 정리한다.

체크	자기 점검 질문	메모
☐	상황이 관찰 가능한가?	
☐	과제가 명확한가?	
☐	제약 조건이 있는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

STAR 작성: 행동과 결과

이 페이지의 결론

Action 과 Result 는 실력의 핵심이다. 내가 어떻게 생각하고 검증했는지가 보여야 한다.

테크 리더의 렌즈

- 행동은 재현, 데이터/로그 확인, 가설, 수정, 검증 순서로 정리한다.
- 결과는 수치, 성공 기준, 재현 가능성, 인사이트 또는 위험 감소로 표현한다.
- 결과가 없으면 배운 점과 후속 조치를 작성한다.

작성 프레임

항목	작성 기준
Action	재현 -> 로그 확인 -> 원인 가설 -> 수정 -> 테스트
Result	배포 성공, 오류 0 건, 로딩 30% 감소, 테스트 통과
후속	체크리스트, 문서화, 모니터링

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
원인을 찾아 고침	네트워크 탭에서 요청 URL 을 확인해 환경변수 누락을 찾고 .env.example 과 배포 문서를 보완.
좋아짐	새 환경에서 문서만 보고 실행에 성공했다.
잘 만들었다.	어떤 문제를 어떤 기준으로 해결했고 어떻게 검증했는지까지 작성한다.
나중에 설명 가능	문서만 보고도 판단할 수 있게 근거 링크와 확인 방법을 기록한다.

수업 중 적용

관련 링크를 README 에 연결해 면접에서 바로 보여줄 수 있게 정리한다.

체크	자기 점검 질문	메모
☐	행동 순서가 드러나는가?	
☐	결과가 확인 가능한가?	
☐	재발 방지 조치가 있는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

결과 수치화

이 페이지의 결론

수치화는 과장이 아니라 검증의 언어이다. 숫자가 어렵다면 확인 방법이라도 기록한다.

테크 리더의 렌즈

- 로딩 시간, 쿼리 수, 오류 재현 여부, 테스트 통과, 배포 성공 여부를 결과로 작성한다.
- 정확한 수치가 없으면 측정 기준, 관찰 결과, 다음 검증 계획을 적다.
- 과장된 성과 표현은 피한다.

작성 프레임

항목	작성 기준
정량	로딩 2.4 초 -> 1.6 초, 쿼리 21 회 -> 3 회
정성	새 사용자 실행 절차에서 환경변수 오류 제거
검증	Lighthouse, 테스트 로그, 배포 로그, 재현 절차

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
성능 개선	초기 렌더링 시간을 Lighthouse 기준 3.2 초에서 2.1 초로 줄였다.
오류 해결	동일 재현 절차에서 401 오류가 더 이상 발생하지 않았다.
잘 만들었다.	어떤 문제를 어떤 기준으로 해결했고 어떻게 검증했는지까지 작성한다.
나중에 설명 가능	문서만 보고도 판단할 수 있게 근거 링크와 확인 방법을 기록한다.

수업 중 적용

관련 링크를 README 에 연결해 면접에서 바로 보여줄 수 있게 정리한다.

체크	자기 점검 질문	메모
☐	측정 가능한 결과가 있는가?	
☐	수치가 없다면 확인 방법이 있는가?	
☐	과장을 피했는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

Frontend 면접 연결

이 페이지의 결론

Frontend 답변은 기술명보다 사용자 경험과 상태 변화에 대한 판단에서 시작해야 한다.

테크 리더의 렌즈

- 상태관리, 렌더링 성능, 접근성, 컴포넌트 분리 질문에 대비한다.
- README 의 구조도와 GIF 를 근거로 답한다.
- 왜 그 방식이 사용자 문제를 줄였는지 설명한다.

작성 프레임

항목	작성 기준
질문	상태 관리를 왜 그렇게 나눴는가?
근거	README 상태관리 섹션, 컴포넌트 다이어그램
답변	문제 -> 선택 -> 검증 -> 한계

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
Redux 사용했다.	서버 상태는 React Query 로 캐싱하고 폼 입력은 로컬 상태로 두어 불필요한 전역 상태를 줄였다.
반응형을 적용 했다.	모바일 폭에서 CTA 버튼 터치 영역과 카드 줄바꿈을 검증했다.
잘 만들었다.	어떤 문제를 어떤 기준으로 해결했고 어떻게 검증했는지까지 작성한다.
나중에 설명 가능	문서만 보고도 판단할 수 있게 근거 링크와 확인 방법을 기록한다.

수업 중 적용

관련 링크를 README 에 연결해 면접에서 바로 보여줄 수 있게 정리한다.

체크	자기 점검 질문	메모
☐	상태관리 선택 이유를 말할 수 있는가?	
☐	화면 흐름 증거가 있는가?	
☐	접근성/성능 질문에 대비했는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

Backend 면접 연결

이 페이지의 결론

Backend 답변은 요청 흐름, 데이터 일관성, 장애 상황을 다루는 능력으로 평가된다.

테크 리더의 렌즈

- API 설계, DB 모델링, 인증/인가, 트랜잭션, 장애 대응 질문에 대비한다.
- Mermaid 요청 흐름과 DB 설계 이유를 연결한다.
- 예외 케이스를 어떻게 다뤘는지가 중요하다.

작성 프레임

항목	작성 기준
질문	이 API 구조를 왜 선택했는가?
근거	요청 흐름도, DB 모델링, 예외 처리 문서
답변	요구사항 -> 구조 -> 트랜잭션/권한 -> 검증

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
API 만들었다.	주문 생성에서 재고 차감과 주문 저장을 하나의 트랜잭션으로 묶어 불일치를 줄였다.
DB 설계	조회 패턴을 고려해 사용자와 주문 관계에 인덱스를 추가했다.
잘 만들었다.	어떤 문제를 어떤 기준으로 해결했고 어떻게 검증했는지까지 작성한다.
나중에 설명 가능	문서만 보고도 판단할 수 있게 근거 링크와 확인 방법을 기록한다.

수업 중 적용

관련 링크를 README 에 연결해 면접에서 바로 보여줄 수 있게 정리한다.

체크	자기 점검 질문	메모
☐	요청 흐름을 설명할 수 있는가?	
☐	DB 설계 근거가 있는가?	
☐	장애/예외 질문에 대비했는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

Security 와 Data 면접 연결

이 페이지의 결론

Security 와 Data 는 근거와 재현성이 곧 실력이다.

테크 리더의 렌즈

- Security 는 위협 모델, 통제, 검증 증거, 완화 후 결과를 연결한다.
- Data 는 Business Question, Data Overview, Methodology, Insight, Proposal 을 연결한다.
- 문서가 구체적일수록 면접 답변은 외운 말이 아니라 증거 기반 설명이 된다.

작성 프레임

항목	작성 기준
Security	위협 시나리오 -> 통제 방식 -> 검증 결과
Data	Business Question -> Data Overview -> Insight -> Proposal
공통	과장보다 근거와 한계

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
보안 검증	관리자 API 는 역할 기반 접근 제어를 적용하고 일반 사용자 요청 실패를 테스트했다.
데이터 분석	결측치 제거 기준과 지표 정의를 문서화해 결과 재현성을 확보했다.
잘 만들었다.	어떤 문제를 어떤 기준으로 해결했고 어떻게 검증했는지까지 작성한다.
나중에 설명 가능	문서만 보고도 판단할 수 있게 근거 링크와 확인 방법을 기록한다.

수업 중 적용

관련 링크를 README 에 연결해 면접에서 바로 보여줄 수 있게 정리한다.

체크	자기 점검 질문	메모
☐	위험과 대응이 연결되는가?	
☐	데이터 처리 기준이 재현 가능한가?	
☐	한계와 개선 방향이 있는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

30 초 프로젝트 소개

이 페이지의 결론

30 초 소개는 프로젝트의 압축 파일이다. 문제, 역할, 선택, 결과가 모두 들어가야 한다.

테크 리더의 렌즈

- 이 프로젝트는 무엇을 해결했고, 나는 어떤 역할을 했으며, 어떤 기술적 선택과 결과가 있었는지 말한다.
- 말로 끝내지 말고 GitHub 의 특정 섹션을 근거로 연결한다.
- 외운 문장보다 구조화된 답변이 좋다.

작성 프레임

항목	작성 기준
문제	누구의 어떤 불편을 해결했는가
역할	내가 맡은 범위와 책임
선택	핵심 기술 판단 1 개
결과	확인 가능한 변화와 링크

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
저는 쇼핑몰을 만들었다.	이 프로젝트는 재고 불일치를 줄이기 위한 주문 관리 서비스이며, 저는 주문 API 와 트랜잭션 처리를 담당했다.
열심히 했다.	배송 오류를 STAR 문서로 정리해 재현 가능한 실행 가이드를 만들었다.
잘 만들었다.	어떤 문제를 어떤 기준으로 해결했고 어떻게 검증했는지까지 작성한다.
나중에 설명 가능	문서만 보고도 판단할 수 있게 근거 링크와 확인 방법을 기록한다.

수업 중 적용

관련 링크를 README 에 연결해 면접에서 바로 보여줄 수 있게 정리한다.

체크	자기 점검 질문	메모
☐	30 초 안에 말할 수 있는가?	
☐	문제-역할-선택-결과가 있는가?	
☐	GitHub 근거 링크가 있는가?	
☐	30 초 안에 말했을 때 어색한 문장이 없는가?	
☐	소개 문장 안에 GitHub 에서 보여줄 대표 링크가 연결되는가?	
☐	기술명 나열보다 문제 해결과 역할이 먼저 나오는가?	

모의면접 피어리뷰

이 페이지의 결론

모의면접의 목적은 답을 맞히는 것이 아니라 문서와 말이 연결되는지 확인하는 것이다.

테크 리더의 렌즈

- 지원자는 30 초 소개 후 직무별 질문 하나에 답한다.
- 면접관 역할은 답변 근거가 GitHub 어디에 있는지 확인한다.
- 근거 없는 답변은 문서 보장 항목이다.

작성 프레임

항목	작성 기준
지원자	30 초 소개 -> 질문 답변 -> GitHub 근거 제시
면접관	추상 표현 표시 -> 근거 링크 요청
피드백	좋은 문장 1개, 보완할 근거 1개

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
제가 했다.	README 의 PR #4 와 STAR 사례에서 제가 수정한 흐름을 확인할 수 있다.
기술적으로 좋다.	캐싱 전략을 선택한 이유와 검증 결과는 Trade-off 표에 정리했다.
열심히 만들었다.	문제, 역할, 기술 선택, 검증 결과를 30 초 안에 말하도록 압축한다.
여러 기능이 있다.	지원 직무와 연결되는 대표 기능 하나를 깊게 설명한다.

수업 중 적용

관련 링크를 README 에 연결해 면접에서 바로 보여줄 수 있게 정리한다.

체크	자기 점검 질문	메모
☐	답변 근거를 GitHub 의 차트, 로그, PR, README 에서 찾았는가?	
☐	추상 표현을 줄였는가?	
☐	보완할 문서 항목이 생겼는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

최종 검증: 연결성

이 페이지의 결론

고급 포트폴리오는 항목이 많은 문서가 아니라 서로 연결된 문서이다.

테크 리더의 렌즈

- README 상단에서 배포, 구조도, 의사결정, STAR, Issue/PR 로 이동할 수 있어야 한다.
- 각 링크에는 무엇을 보여주는지 설명이 있어야 한다.
- 면접관이 길을 잃지 않도록 흐름을 설계한다.

작성 프레임

항목	작성 기준
상단	프로젝트 목적과 대표 링크
본문	실행, 구조, 의사결정, 트러블슈팅
하단	협업 기록, 향후 개선, 참고 자료
연결	링크와 짧은 설명

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
링크 모음	배포, 아키텍처, STAR 사례, 대표 PR 을 각각 한 문장 설명과 함께 배치한다.
문서 많음	읽는 순서와 추천 링크를 README 상단에 제공한다.
잘 만들었다.	어떤 문제를 어떤 기준으로 해결했고 어떻게 검증했는지까지 작성한다.
나중에 설명 가능	문서만 보고도 판단할 수 있게 근거 링크와 확인 방법을 기록한다.

수업 중 적용

관련 링크를 README 에 연결해 면접에서 바로 보여줄 수 있게 정리한다.

체크	자기 점검 질문	메모
☐	README 에서 핵심 링크로 이동 가능한가?	
☐	각 링크 설명이 있는가?	
☐	STAR 와 PR 이 연결되는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

2 일차 마무리와 최종 제출

이 페이지의 결론

이제 GitHub 는 코드 창고가 아니라 면접에서 꺼내 쓸 수 있는 기술 증거가 되어야 한다.

테크 리더의 렌즈

- README 상단, 배포/실행, 직무별 기술 증거, Mermaid, Issue/PR, STAR, 30 초 소개를 다시 점검한다.
- 대표 링크 3 개를 개발 증거, 데이터 증거, 보안 증거 중 지원 직무에 맞게 고르면 면접 답변이 훨씬 안정된다.
- 앞으로 프로젝트를 고칠 때마다 왜 바꿨는지 GitHub 에 기록한다.

작성 프레임

항목	작성 기준
대표 링크 1	배포 또는 실행 가이드
대표 링크 2	아키텍처/Mermaid 또는 Trade-off
대표 링크 3	STAR 또는 대표 PR
다음 작업	수업 후 개선 Issue 1 개

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
완성했다.	대표 링크 3 개와 다음 개선 Issue 까지 준비한다.
코드만 제출	README 와 면접 답변까지 연결해야 포트폴리오가 된다.
잘 만들었다.	어떤 문제를 어떤 기준으로 해결했고 어떻게 검증했는지까지 작성한다.
나중에 설명 가능	문서만 보고도 판단할 수 있게 근거 링크와 확인 방법을 기록한다.

수업 중 적용

관련 링크를 README 에 연결해 면접에서 바로 보여줄 수 있게 정리한다.

체크	자기 점검 질문	메모
☐	최종 검증표를 모두 확인했는가?	
☐	대표 링크 3 개를 골랐는가?	
☐	다음 개선 Issue 를 만들었는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, PR 중 하나로 확인 가능한가?	

데이터·보안 특화 보강 로드맵

이 페이지의 결론
개발 직군이 아니어도 GitHub 는 사고 과정과 증거를 보여주는 포트폴리오가 될 수 있다.

테크 리더의 렌즈

- 데이터 학생은 인사이트와 비즈니스 제안을 중심에 둡니다.
- 보안 학생은 위협 식별, 검증, 완화 과정을 중심에 둡니다.
- 두 직군 모두 결과보다 판단 근거와 재현 가능한 증거가 중요하다.

작성 프레임

항목	작성 기준
데이터	Business Question -> Data Overview -> Methodology -> Insight -> Proposal
보안	Executive Summary -> Threat Model -> Process -> Evidence -> Lesson Learned
공통	문제, 근거, 선택, 결과, 다음 행동이 README 에서 연결되어야 한다.

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
데이터 분석 프로젝트이다.	지역별 매출 편차의 원인을 공공·상권 데이터로 분석하고 입지 전략을 제안했다.
보안 공부했다.	인증 흐름의 브루트 포스 위험을 가정하고 Rate Limiting 적용 전후 로그를 비교했다.
차트 넣었다.	핵심 인사이트 3 개를 차트와 함께 배치하고, 각 차트가 답하는 질문을 명시했다.
툴 사용 가능	Wireshark 는 평문 전송 구간 식별, Nmap 은 노출 포트 확인 목적으로 사용했다.

수업 중 적용

본인 프로젝트가 데이터형, 보안형, 개발형 중 어디에 가까운지 표시하고 README 목차를 다시 잡는다.

체크	자기 점검 질문	메모
☐	내 프로젝트의 지원 직무가 분명한가?	
☐	README 첫 화면에서 직무 적합성이 보이는가?	
☐	분석 과정과 증거가 기능 설명보다 앞에 오는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, 로그 중 하나로 확인 가능한가?	

데이터 README 의 첫 질문 설계

이 페이지의 결론

데이터 포트폴리오는 '무엇을 분석했는가'보다 '왜 이 분석이 필요한가'에서 수준이 갈린다.

테크 리더의 렌즈

- Business Question 은 채용 담당자가 프로젝트의 맥락을 이해하는 입구이다.
- 질문은 데이터로 답할 수 있어야 하며, 결론은 행동 제안으로 이어져야 한다.
- 좋은 질문은 도메인, 대상, 의사결정 상황을 함께 담는다.

작성 프레임

항목	작성 기준
문제 배경	누가 어떤 의사결정에서 어려움을 겪는가?
분석 질문	데이터로 답할 수 있는 구체 질문 1 개를 작성한다.
성공 기준	인사이트가 어떤 행동을 바꾸면 성공인가를 정한다.
제약	기간, 데이터 품질, 표본 한계를 미리 밝힙니다.

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
매출 데이터를 분석했다.	월별 매출 하락의 원인이 방문자 감소인지 객단가 변화인지 구분하기 위해 주문 데이터를 분석했다.
상권 분석	신규 매장 후보지를 고르기 위해 유동 인구, 임대료, 경쟁 점포 밀도를 비교했다.
공공데이터 사용	청년 정책 수요가 높은 지역을 찾기 위해 인구·교통·시설 데이터를 결합했다.
왜 했는지 없음	분석 목적을 '누구의 어떤 결정에 도움을 주는가'로 다시 작성한다.

수업 중 적용

README 상단에 Business Question 1 문장과 이 질문이 중요한 이유 2 문장을 추가한다.

체크	자기 점검 질문	메모
☐	질문이 데이터로 답할 수 있는 형태인가?	
☐	분석 대상과 의사결정자가 드러나는가?	
☐	결과가 제안으로 이어지는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, 로그 중 하나로 확인 가능한가?	

Data Overview 를 신뢰 문서로 만들기

이 페이지의 결론
데이터 설명은 단순 출처 나열이 아니라, 분석 결과를 믿어도 되는지 판단하게 하는 장치이다.

테크 리더의 렌즈

- 출처, 기간, 규모, 주요 컬럼, 병합 기준을 짧고 정확하게 작성한다.
- 누락, 이상치, 편향 가능성은 숨기지 말고 관리한 방식까지 적다.
- 데이터 사진이나 샘플 이미지는 README 의 신뢰도를 크게 높인다.

작성 프레임

항목	작성 기준
출처	기관명, 수집 경로, 라이선스 또는 공개 범위를 적다.
규모	행 수, 기간, 단위, 주요 컬럼을 숫자로 제시한다.
품질	결측치, 중복, 이상치, 표본 편향을 어떻게 점검했는지 작성한다.
한계	결론을 일반화할 수 없는 범위를 명시한다.

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
데이터는 Kaggle 에서 받음	Kaggle 공개 데이터 12 만행을 사용했고, 2024 년 거래 내역과 상품 카테고리 컬럼을 분석했다.
전처리했다.	결측률 30% 이상 컬럼은 제외하고, 핵심 변수 결측은 도메인 기준으로 대체했다.
이상치 제거	IQR 기준으로 극단값을 탐지하되, 실제 고액 거래는 별도 검토 후 유지했다.
데이터 많음	100 만 건 규모보다 분석 질문에 필요한 컬럼과 기간을 먼저 설명한다.

수업 중 적용

README 에 데이터 출처·규모·품질·한계를 4 줄 표로 추가한다.

체크	자기 점검 질문	메모
☐	데이터 출처와 기간이 명확한가?	
☐	행 수와 주요 컬럼이 숫자로 제시되는가?	
☐	결측치와 이상치 처리 기준이 설명되는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, 로그 중 하나로 확인 가능한가?	

Methodology: 분석 과정을 실력으로 보이게 하기

이 페이지의 결론
Pandas 를 썼다는 말보다, 왜 그 처리와 분석 방법을 선택했는지가 훨씬 강한 증거이다.

테크 리더의 렌즈

- 전처리, EDA, 통계 검정, 시각화, 모델링을 순서대로 연결한다.
- 각 단계마다 선택 이유와 대안을 짧게 기록한다.
- 분석 과정은 재현 가능한 노트북, SQL, 대시보드 링크로 연결한다.

작성 프레임

항목	작성 기준
전처리	중복 제거, 결측 대체, 이상치 처리의 기준을 작성한다.
EDA	분포, 추세, 상관관계, 그룹 차이를 확인한다.
분석	가설, 지표, 비교 기준, 검정 또는 모델 선택 이유를 기록한다.
재현	노트북, SQL, 대시보드, 데이터 사전 링크를 둡니다.

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
Pandas 로 분석	요일별 대여량 차이를 확인하기 위해 groupby 집계 후 히트맵으로 시간대 패턴을 비교했다.
그래프 그림	상관관계 히트맵에서 강한 변수 3 개를 선별하고 해석 가능한 지표로 다시 설명했다.
모델 사용	정확도보다 수요 예측의 과소 추정을 줄이기 위해 MAE 와 피크 시간 오차를 함께 봤다.
전처리 완료	왜 제거했고 무엇이 남았는지를 기준과 숫자로 적다.

수업 중 적용
분석 과정 섹션을 '전처리 -> EDA -> 결과 도출 -> 재현 링크' 순서로 다시 정리한다.

체크	자기 점검 질문	메모
☐	각 단계의 선택 이유가 있는가?	
☐	분석 순서가 읽는 사람에게 자연스러운가?	
☐	노트북이나 SQL 링크로 재현 가능한가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, 로그 중 하나로 확인 가능한가?	

Key Insights 3 개를 뽑는 법

이 페이지의 결론
데이터 프로젝트의 핵심은 차트 개수가 아니라 의사결정을 바꾸는 인사이트 3 개이다.

테크 리더의 렌즈

- 각 인사이트는 관찰, 해석, 행동 제안으로 구성한다.
- 숫자 없는 인사이트는 인상평처럼 보이기 쉽다.
- 차트 제목은 '무엇을 보여주는가'가 아니라 '무엇을 발견했는가'로 작성한다.

작성 프레임

항목	작성 기준
관찰	데이터에서 보인 패턴을 숫자로 작성한다.
해석	그 패턴이 왜 중요한지 도메인 맥락을 붙인다.
제안	그래서 어떤 행동을 해야 하는지 연결한다.
근거	차트, SQL, 노트북 셀, 대시보드 링크를 둡니다.

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
대여량이 많다	출근 시간 08:00~09:00 에 A 지역 대여량이 평균 대비 70% 높았다.
고객이 줄었다	방문자 수보다 재구매율 하락이 매출 감소의 주요 원인으로 나타났다.
상관관계 있음	강수량과 이용량은 음의 상관을 보였지만 주말 효과를 통제하면 영향이 약해졌다.
그래프 참고	차트 아래에 '이 차트로 내린 판단'을 한 문장으로 붙인다.

수업 중 적용
README 의 결과 섹션을 핵심 인사이트 3 개와 각 인사이트의 제안 1 개로 바꾼다.

체크	자기 점검 질문	메모
☐	각 인사이트에 숫자가 포함되어 있는가?	
☐	인사이트마다 행동 제안이 있는가?	
☐	차트와 문장이 서로 같은 메시지를 말하는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, 로그 중 하나로 확인 가능한가?	

Actionable Proposal 로 마무리하기

이 페이지의 결론

데이터 분석의 끝은 대시보드가 아니라 '그래서 무엇을 해야 하는가'에 대한 제안이다.

테크 리더의 렌즈

- 제안은 실행 주체, 실행 방법, 기대 효과, 검증 방법을 포함한다.
- 학생 프로젝트라도 실무 의사결정 언어로 마무리할 수 있다.
- 좋은 제안은 분석 한계와 다음 실험 계획까지 담는다.

작성 프레임

항목	작성 기준
제안	분석 결과를 바탕으로 바꿀 행동을 1 문장으로 작성한다.
근거	인사이트 번호와 차트 링크를 연결한다.
효과	기대되는 변화와 측정 지표를 적다.
다음 검증	A/B 테스트, 추가 데이터, 현장 검증 계획을 작성한다.

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
재배치 필요	출근 피크 30 분 전 A 구역에 자전거를 선배치하고, 품질률을 전주 대비로 비교한다.
마케팅 강화	재구매율이 낮은 세그먼트에 쿠폰보다 온보딩 메시지를 우선 실험한다.
대시보드 제작	운영자가 매일 볼 지표 4 개를 정의하고 이상 신호 알림 기준을 추가했다.
좋은 결과	좋았다는 말 대신 어떤 지표가 얼마나 개선될 수 있는지 가설을 작성한다.

수업 중 적용

분석 결론 아래에 실행 제안 2 개와 검증 지표 1 개를 추가한다.

체크	자기 점검 질문	메모
☐	제안의 실행 주체가 보이는가?	
☐	기대 효과가 지표로 연결되는가?	
☐	다음 검증 방법이 있는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, 로그 중 하나로 확인 가능한가?	

데이터 시각화 증거 배치

이 페이지의 결론

시각화는 장식이 아니라 읽는 사람이 10 초 안에 결론을 이해하게 하는 증거이다.

테크 리더의 렌즈

- README 상단에는 대표 차트 1 개, 결과 섹션에는 근거 차트 2~3 개를 배치한다.
- 각 차트에는 질문, 발견, 해석을 짧게 붙인다.
- Power BI, Tableau, Excel, Python 차트 모두 목적과 해석이 있어야 한다.

작성 프레임

항목	작성 기준
대표 차트	프로젝트의 핵심 결론을 한 장으로 보여준다.
보조 차트	세그먼트, 추세, 예외 사례를 설명한다.
캡션	차트가 답하는 질문과 발견을 함께 작성한다.
링크	대시보드, 이미지 원본, 노트북 셀을 연결한다.

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
차트 첨부	출근 시간대 대여소 포화가 반복되는 상위 10 개 지점을 히트맵으로 제시했다.
대시보드 링크	Tableau 대시보드 링크와 함께 면접관이 먼저 볼 필터 2 개를 안내했다.
상관관계 히트맵	상관이 높은 변수만 고른 뒤 인과로 해석하지 않는다는 한계를 적었다.
이미지만 있음	이미지 아래에 '이 차트에서 무엇을 판단했는가'를 반드시 적다.

수업 중 적용

대표 차트 1 개를 README 상단에 배치하고, 차트 캡션을 판단 문장으로 바꾼다.

체크	자기 점검 질문	메모
☐	대표 차트가 첫 화면에서 보이는가?	
☐	차트마다 질문과 해석이 있는가?	
☐	도구 이름보다 시각화 목적이 먼저 보이는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, 로그 중 하나로 확인 가능한가?	

데이터 프로젝트 주제 확장

이 페이지의 결론
좋은 데이터 주제는 도메인 관심과 분석 역량을 동시에 보여준다.

테크 리더의 렌즈

- 마케팅, 공공, 금융, 유통, 교육 데이터는 학생 프로젝트로도 설득력이 높다.
- 주제는 거창할 필요가 없지만 질문과 제안은 구체적이어야 한다.
- 각 주제는 데이터 출처와 활용 시나리오가 함께 있어야 한다.

작성 프레임

항목	작성 기준
마케팅	고객 세그먼트, 이탈, 캠페인 반응, 재구매 분석
공공	교통, 상권, 복지, 환경, 시설 접근성 분석
금융	소비 패턴, 이상 거래 탐지, 리스크 지표 분석
유통	재고, 수요 예측, 가격, 매장 입지 분석

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
마케팅 데이터	캠페인 클릭보다 전환 이후 재구매율을 기준으로 고객 세그먼트를 재정의했다.
공공 데이터	청년 시설 접근성이 낮은 지역을 교통 시간과 인구 비중으로 비교했다.
금융 데이터	이상 거래 후보를 규칙 기반으로 탐지하고 오탐 가능성을 별도 검토했다.
유통 데이터	품질률이 높은 SKU 와 요일 패턴을 연결해 발주 기준을 제안했다.

수업 중 적용

본인 관심 도메인 하나를 골라 Business Question, 데이터 출처, 예상 제안을 작성한다.

체크	자기 점검 질문	메모
☐	관심 도메인이 README 에서 분명한가?	
☐	데이터 출처를 실제로 확보할 수 있는가?	
☐	주제가 지원 직무와 연결되는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, 로그 중 하나로 확인 가능한가?	

데이터 README 실전 예시

이 페이지의 결론

예시는 그대로 베끼는 템플릿이 아니라, 본인 프로젝트를 구조화하는 기준표이다.

테크 리더의 렌즈

- 자전거 대여, 상권 분석, 이탈 예측처럼 누구나 이해할 수 있는 문제로 시작한다.
- 데이터 구성과 분석 과정은 짧게, 인사이트와 제안은 선명하게 작성한다.
- 기술 스택은 '무엇을 위해 썼는가'와 함께 적다.

작성 프레임

항목	작성 기준
제목	2025 서울시 자전거 대여 데이터 기반 이용 패턴 분석
데이터	대여 이력 100 만 건, 날씨 데이터 병합, 시간대 기준 집계
분석	요일·시간대 히트맵, 편도 이동 패턴, 포화 구간 탐지
제안	재배치 시작 시간 30 분 조정, 우선 구역 지정

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
Tableau 사용	Tableau 로 대여소별 포화 패턴을 시각화하고 운영자 필터를 설계했다.
SQL 사용	대여 이력에서 출근 시간대 이동량을 추출하기 위해 시간대별 집계 쿼리를 작성했다.
Excel 사용	초기 품질 점검과 피벗 분석으로 이상값 후보를 선별했다.
Python 사용	Pandas 로 병합과 전처리를 수행하고 Seaborn 으로 패턴을 검증했다.

수업 중 적용

README 에 '분석 배경-데이터 구성-분석 과정-핵심 인사이트-제안' 목차를 적용한다.

체크	자기 점검 질문	메모
☐	예시 구조를 내 프로젝트에 맞게 바꾸었는가?	
☐	기술 스택이 목적과 함께 설명되는가?	
☐	제안이 분석 결과에서 자연스럽게 나오는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, 로그 중 하나로 확인 가능한가?	

데이터 직군 면접 답변 연결

이 페이지의 결론
README 는 면접에서 '왜 그렇게 분석했는가?'라는 질문에 답하기 위한 증거 묶음이다.

테크 리더의 렌즈

- 면접 질문은 분석 목적, 데이터 품질, 방법 선택, 한계, 제안에서 나옵니다.
- 각 답변은 README 의 특정 차트나 노트북 셀로 연결되어야 한다.
- 모르는 영역은 숨기지 말고 다음 검증 계획으로 전환한다.

작성 프레임

항목	작성 기준
질문	왜 이 데이터를 선택했는가?
답변	의사결정 문제와 연결해 데이터 선택 이유를 설명한다.
근거	README 의 Data Overview, EDA, 차트 링크를 보여준다.
확장	추가 데이터가 있다면 어떻게 검증할지 말한다.

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현	
그냥 공개 데이터라서	분석 질문에 필요한 시간대·지역·이용량 변수가 있어 해당 데이터를 선택했다.	
전처리했다.	결측률과 이상치 기준을 먼저 정하고 결과 왜곡 가능성을 줄였다.	
모델 성능 좋음	정확도뿐 아니라 피크 시간대 오차를 따로 확인해 운영 관점에서 평가했다.	
한계 없음	날씨 외 이벤트 변수는 반영하지 못해 후속 분석에서 보완할 계획이다.	
수업 중 적용 데이터 프로젝트 예상 면접 질문 5 개를 만들고, 각 질문 옆에 README 근거 링크를 적다.		
체크	자기 점검 질문	메모
☐	분석 목적을 30 초 안에 말할 수 있는가?	
☐	데이터 품질 질문에 답할 수 있는가?	
☐	한계와 다음 계획을 준비했는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, 로그 중 하나로 확인 가능한가?	

보안 README 의 중심축: 위협 탐지와 완화

이 페이지의 결론

보안 포트폴리오는 기능 구현보다 어떤 위협을 식별하고 어떻게 검증했는지를 보여줄 때 강하다.

테크 리더의 렌즈

- Executive Summary 에서 보안적 동기와 발견 사항을 먼저 요약한다.
- Threat Model 은 공격자, 자산, 공격 경로, 방어 목표를 정리한다.
- Evidence & Findings 는 로그, 요청, 스크린샷, 설정 변화로 증명한다.

작성 프레임

항목	작성 기준
요약	무엇을 보호하려 했고 무엇을 발견했는가?
위협 모델	공격자, 자산, 공격 경로, 영향 범위를 정의한다.
검증 과정	도구, 시나리오, 테스트 조건을 기록한다.
완화	적용한 정책과 차단 결과를 증거로 보여준다.

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
로그인 기능 구현	인증 프로세스의 브루트 포스 위협을 가정하고 Rate Limiting 전후 실패 로그를 비교했다.
보안 검증	관리자 API 에 역할 기반 접근 제어를 적용하고 일반 사용자 요청 실패를 검증했다.
취약점 찾을	입력값 검증 누락으로 XSS 가능성을 확인하고 필터링 정책을 설계했다.
툴 사용	도구 이름보다 어떤 위협을 검증했는지를 먼저 작성한다.

수업 중 적용

README 첫 문단을 기능 소개에서 보안 문제와 검증 목표 중심으로 바꾼다.

체크	자기 점검 질문	메모
☐	보호하려는 자산이 분명한가?	
☐	위협과 영향이 연결되어 있는가?	
☐	완화 전후 증거가 있는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, 로그 중 하나로 확인 가능한가?	

Threat Model 작성법

이 페이지의 결론

위협 모델은 보안 프로젝트를 ‘해봤다’가 아니라 ‘생각하고 검증했다’로 바뀌주는 핵심 문서이다.

테크 리더의 렌즈

- 공격자를 먼저 정하면 테스트 범위가 선명해집니다.
- 자산과 공격 경로를 구분하면 방어 전략이 논리적으로 보인다.
- 위협 모델은 복잡한 도식보다 명확한 표 하나가 더 효과적일 때가 많다.

작성 프레임

항목	작성 기준
공격자	비로그인 사용자, 일반 사용자, 내부 사용자 등 가정
자산	계정, 개인정보, 관리자 API, 결제 정보, 로그
경로	입력값, 인증, 세션, 네트워크, 파일 업로드
영향	권한 상승, 정보 노출, 서비스 중단, 데이터 변조

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
해킹 위험 있음	일반 사용자가 관리자 API 를 호출해 권한 상승을 시도할 수 있다고 가정했다.
XSS 가능	댓글 입력값이 HTML 로 렌더링되는 경로를 공격 표면으로 정의했다.
네트워크 보안	로그인 요청이 평문으로 전송될 경우 계정 정보 노출 가능성을 점검했다.
막연한 위협	누가 무엇을 어떻게 공격하는지 네 칸 표로 구체화한다.

수업 중 적용

본인 프로젝트에 대한 공격자·자산·공격 경로·영향 표를 README 에 추가한다.

체크	자기 점검 질문	메모
☐	공격자 가정이 현실적인가?	
☐	보호 자산이 구체적으로 적혔는가?	
☐	공격 경로와 완화 방법이 연결되는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, 로그 중 하나로 확인 가능한가?	

Evidence & Findings 설계

이 페이지의 결론

보안 보고서는 증거가 생명이다. 말이 아니라 로그와 화면으로 보여줘야 한다.

테크 리더의 렌즈

- 공격 시도, 취약 조건, 방어 후 차단 결과를 한 흐름으로 배치한다.
- 스크린샷은 무엇을 증명하는지 캡션이 있어야 한다.
- 민감 정보는 반드시 마스킹하고, 공격 재현 정보는 교육 목적 범위로 제한한다.

작성 프레임

항목	작성 기준
공격 전	위협 가정과 테스트 요청을 보여준다.
발견	로그, 응답 코드, 화면, 코드 구간으로 근거를 제시한다.
조치	설정, 검증 로직, 정책 변경 내용을 작성한다.
공격 후	차단 결과와 남은 한계를 기록한다.

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
캡처 있음	Burp 요청 캡처 아래에 어떤 검증값이 누락되었는지 설명했다.
로그 분석	실패 로그인 5 회 이후 429 응답이 발생하는 로그를 첨부했다.
코드 수정	입력값 검증 함수와 테스트 케이스 링크를 함께 연결했다.
증거 없음	보안 주장은 반드시 로그, 요청, 화면, 테스트 중 하나로 뒷받침한다.

수업 중 적용

보안 README 에 공격 전/발견/조치/공격 후 증거 4 칸 표를 만든다.

체크	자기 점검 질문	메모
☐	증거가 시간 순서대로 배치되어 있는가?	
☐	스크린샷 캡션이 판단 문장인가?	
☐	민감 정보가 마스킹되어 있는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, 로그 중 하나로 확인 가능한가?	

보안 도구는 목적과 함께 적기

이 페이지의 결론
Wireshark, Nmap, Burp Suite 는 이름보다 무엇을 검증했는지가 더 중요하다.

테크 리더의 렌즈

- 도구 목록은 숙련도를 보장하지 않는다. 사용 목적과 결과를 같이 작성한다.
- 도구 결과는 위험, 영향, 조치로 번역되어야 한다.
- 합법적 실습 환경과 본인 프로젝트 범위 안에서 수행했다는 점을 분명히 한다.

작성 프레임

항목	작성 기준
Wireshark	패킷 분석, 평문 전송 확인, 프로토콜 흐름 점검
Nmap	노출 포트, 서비스 버전, 불필요한 표면 확인
Burp Suite	요청 변조, 인증 흐름, 입력값 검증 테스트
로그 도구	공격 시도 탐지, 차단 정책 검증, 알림 기준 확인

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
Wireshark 사용	로그인 요청의 전송 구간을 캡처해 민감 정보가 평문으로 노출되지 않는지 확인했다.
Nmap 사용	배포 서버에서 의도하지 않은 포트 노출 여부를 점검하고 방화벽 규칙을 정리했다.
Burp 사용	요청 파라미터 변조로 권한 검증 누락 가능성을 확인했다.
Metasploit 사용	허가된 실습 환경과 학습 목적을 명시하고 결과를 방어 관점으로 정리한다.

수업 중 적용
사용 도구 목록을 '도구-검증 목적-확인 결과-조치' 표로 바꾼다.

체크	자기 점검 질문	메모
☐	도구마다 사용 목적이 적혀 있는가?	
☐	결과가 위험과 조치로 번역되는가?	
☐	실습 범위와 윤리 기준이 명시되어 있는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, 로그 중 하나로 확인 가능한가?	

웹 보안 프로젝트 README

이 페이지의 결론
웹 해킹 주제는 취약점 이름보다 인증, 입력, 권한, 세션 흐름을 어떻게 검증했는지가 중요하다.

테크 리더의 렌즈

- XSS, SQL Injection, 인증 우회, Rate Limiting 은 학생 프로젝트에서도 설명하기 좋다.
- 공격 재현보다 방어 설계와 검증 결과를 중심에 둡니다.
- 취약점 설명은 교육 목적과 본인 환경 범위로 제한한다.

작성 프레임

항목	작성 기준
XSS	입력값 검증, 출력 인코딩, 차단 화면
인증	브루트 포스, 세션 만료, 비밀번호 정책
권한	일반 사용자와 관리자 API 접근 차이
검증	요청 캡처, 응답 코드, 테스트 케이스, 로그

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
XSS 테스트	댓글 입력값이 렌더링되는 위치를 점검하고 출력 인코딩 적용 후 실행이 차단됨을 확인했다.
Rate Limiting	동일 IP 에서 5 회 실패 후 429 응답과 잠금 로그가 남도록 검증했다.
권한 검증	일반 사용자의 관리자 API 호출이 403 으로 차단되는 테스트를 추가했다.
취약점 공부	공격명만 나열하지 말고 내 프로젝트의 어느 흐름에서 검증했는지 작성한다.

수업 중 적용
본인 프로젝트의 인증·입력·권한 중 하나를 골라 위협과 완화 내용을 README 에 추가한다.

체크	자기 점검 질문	메모
☐	취약점이 프로젝트 흐름과 연결되는가?	
☐	방어 전후 차이가 증거로 보이는가?	
☐	공격 세부 내용이 과도하게 노출되지 않는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, 로그 중 하나로 확인 가능한가?	

네트워크 분석 프로젝트 README

이 페이지의 결론
네트워크 보안 포트폴리오에 패킷을 봤다는 사실보다 트래픽에서 어떤 위험 신호를 읽었는지가 핵심이다.

테크 리더의 렌즈

- 평균 전송, 불필요한 포트, 비정상 요청 패턴은 좋은 분석 주제이다.
- 패킷 캡처는 흐름, 프로토콜, 위험, 대응으로 해석해야 한다.
- 실습 캡처 파일은 민감 정보 제거와 범위 설명이 필요하다.

작성 프레임

항목	작성 기준
관찰	트래픽 흐름, 포트, 프로토콜, 요청 빈도
위험	정보 노출, 공격 표면, 비정상 패턴
검증	캡처 필터, 로그 비교, 재현 조건
대응	HTTPS, 방화벽, 접근 제한, 모니터링 기준

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
패킷 분석	Wireshark 로 로그인 요청 흐름을 확인하고 민감 정보가 암호화 구간에서 전송됨을 검증했다.
포트 스캔	Nmap 결과에서 개발용 포트 노출을 발견하고 배포 설정에서 접근을 제한했다.
이상 트래픽	짧은 시간 반복 요청을 로그와 비교해 Rate Limiting 기준을 제안했다.
네트워크 공부	캡처 화면만 넣지 말고 어떤 위험을 읽었는지 문장으로 적다.

수업 중 적용
네트워크 분석 결과를 '관찰-위험-검증-대응' 4 단계로 README 에 정리한다.

체크	자기 점검 질문	메모
☐	트래픽 관찰이 위험 해석으로 이어지는가?	
☐	캡처 범위와 조건이 명확한가?	
☐	민감 정보가 제거되어 있는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, 로그 중 하나로 확인 가능한가?	

보안 커뮤니티와 학습 흔적

이 페이지의 결론
보안 관심자는 꾸준히 배우고 윤리적으로 행동한다는 신뢰를 함께 보여줘야 한다.

테크 리더의 렌즈

- CTF, 워게임, 보안 뉴스 분석, 자격증 준비는 학습 지속성을 보여준다.
- 단순 참여 기록보다 배운 점과 프로젝트에 적용한 점이 중요하다.
- 공격 기술은 책임 있는 공개와 허가된 환경 원칙을 함께 적다.

작성 프레임

항목	작성 기준
CTF	문제 유형, 접근 방식, 배운 취약점 개념
뉴스 분석	취약점 이슈, 영향 범위, 내 프로젝트 점검 항목
자격증	학습 중인 영역과 프로젝트 연결 지점
윤리	허가된 환경, 민감 정보 보호, 공개 범위 원칙

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
CTF 참여	XSS 문제를 풀며 출력 인코딩의 중요성을 이해했고 댓글 기능 검증에 적용했다.
뉴스레터 구독	최근 인증 우회 사례를 읽고 내 프로젝트의 토큰 만료 정책을 점검했다.
자격증 공부	네트워크 기본기를 보완하기 위해 패킷 흐름과 포트 노출 점검을 병행했다.
공부 중	학습 흔적은 프로젝트 개선으로 연결될 때 포트폴리오 가치가 커집니다.

수업 중 적용
README 하단에 보안 관심 분야, 학습 흔적, 프로젝트 적용 내용을 3 줄로 추가한다.

체크	자기 점검 질문	메모
☐	학습 기록이 프로젝트 개선과 연결되는가?	
☐	관심 분야가 구체적인가?	
☐	윤리적 실습 범위가 명시되어 있는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, 로그 중 하나로 확인 가능한가?	

데이터·보안 STAR 답변 구조

이 페이지의 결론

STAR 는 경험을 자랑하는 틀이 아니라 증거를 면접 답변으로 압축하는 도구이다.

테크 리더의 렌즈

- Situation 은 문제 배경, Task 는 판단해야 할 과제이다.
- Action 은 분석 또는 검증 과정, Result 는 숫자·증거·제안으로 말한다.
- 데이터와 보안 모두 GitHub 링크가 답변의 근거가 되어야 한다.

작성 프레임

항목	작성 기준
Situation	어떤 데이터 문제 또는 보안 위험을 발견했는가?
Task	무엇을 판단하거나 검증해야 했는가?
Action	어떤 기준과 도구로 분석 또는 검증했는가?
Result	무엇을 발견했고 어떤 제안 또는 완화를 했는가?

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
데이터 STAR	상권 매출 하락 원인을 방문자 수와 객단가로 분해해 재방문 캠페인 우선순위를 제안했다.
보안 STAR	브루트 포스 위험을 가정하고 실패 횟수 제한 정책을 적용한 뒤 429 로그로 차단을 확인했다.
열심히 함	행동보다 판단 기준과 결과 증거를 먼저 말한다.
팀 프로젝트	내 역할, 내가 만든 분석 기준, 내가 검증한 증거를 분리해 작성한다.

수업 중 적용

데이터 또는 보안 프로젝트 1 개를 STAR 4 문장으로 요약하고 README 링크를 붙인다.

체크	자기 점검 질문	메모
☐	문제와 과제가 구분되어 있는가?	
☐	Action 에 도구와 기준이 함께 있는가?	
☐	Result 가 증거와 제안으로 끝나는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, 로그 중 하나로 확인 가능한가?	

직무별 대표 링크 3 개 구성

이 페이지의 결론

면접관에게 보여줄 링크는 많을수록 좋은 것이 아니라 직무 적합성을 증명하는 3 개면 충분하다.

테크 리더의 렌즈

- 데이터 직군은 README, 대시보드, 노트북 또는 SQL 링크가 좋다.
- 보안 직군은 Threat Model, Evidence, 검증 로그 또는 테스트 링크가 좋다.
- 각 링크에는 무엇을 보면 되는지 한 줄 설명이 필요하다.

작성 프레임

항목	작성 기준
링크 1	프로젝트 README: 문제, 과정, 결과 전체 흐름
링크 2	핵심 증거: 대시보드, 로그, 차트, 테스트 결과
링크 3	판단 근거: 노트북, SQL, Threat Model, PR
설명	각 링크가 어떤 역할을 증명하는지 작성한다.

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
데이터 링크	README 상단 -> Tableau 대시보드 -> 전처리 노트북 순서로 배치한다.
보안 링크	Threat Model -> Burp 요청 캡처 -> 방어 후 테스트 로그 순서로 배치한다.
링크 모음	링크 옆에 '이 링크로 확인할 수 있는 역량'을 붙인다.
깃허브 주소만	대표 링크 3 개를 고르고 나머지는 보조 자료로 내립니다.

수업 중 적용

본인 README 상단에 지원 직무 기준 대표 링크 3 개와 설명 1 줄씩을 배치한다.

체크	자기 점검 질문	메모
☐	대표 링크 3 개가 첫 화면에서 보이는가?	
☐	각 링크의 증명 역량이 다르게 구성되는가?	
☐	면접 답변과 링크가 연결되는가?	
☐	이 항목이 README 의 어느 위치에 반영되어 있는가?	
☐	면접관이 이 내용을 보고 추가 질문을 할 수 있는가?	
☐	증거 링크, 명령어, 화면, 로그 중 하나로 확인 가능한가?	

최종 제출 체크

이 페이지의 결론

최종 포트폴리오는 개발, 데이터, 보안 중 본인의 강점이 한눈에 읽혀야 한다.

테크 리더의 렌즈

- README 첫 화면은 목적, 역할, 결과, 대표 링크를 보여줘야 한다.
- 데이터형은 질문, 데이터, 인사이트, 제안이 이어져야 한다.
- 보안형은 위협, 검증, 증거, 완화가 이어져야 한다.

작성 프레임

항목	작성 기준
공통	README, 실행, 배포, Issue/PR, STAR 가 한 이야기로 연결
데이터	Business Question, Data Overview, Methodology, Insight, Proposal
보안	Executive Summary, Threat Model, Evidence, Mitigation, Lesson Learned
제출	대표 링크 3 개, 30 초 소개, 예상 질문 5 개 준비

아쉬운 표현을 고급 표현으로 바꾸기

아쉬운 표현	개선된 표현
완성했다.	대표 링크 3 개와 예상 질문 5 개까지 준비했을 때 제출 가능한 상태이다.
README 가 길다.	긴 설명보다 면접관이 따라갈 수 있는 탐색 순서가 중요하다.
데이터/보안 둘 다	두 관심사를 모두 쓰되 지원 직무 기준으로 첫 화면 우선순위를 정한다.
나중에 수정	제출 전에는 실행 링크, 캡처, 로그, 차트가 모두 열리는지 확인한다.

수업 중 적용

수업 후 30 분 안에 README 상단, 대표 링크 3 개, STAR 답변 1 개를 최종 점검한다.

체크	자기 점검 질문	메모
☐	지원 직무별 강점이 첫 화면에서 보이는가?	
☐	데이터 또는 보안 증거가 구체적인가?	
☐	대표 링크 3 개가 모두 열리는가?	
☐	30 초 소개가 README 흐름과 같은가?	
☐	예상 질문 5 개와 근거 링크가 준비되었는가?	
☐	수업 후 개선 Issue 를 1 개 만들었는가?	

직무 심화 파트 사용법

이 페이지의 결론

이 파트는 체크리스트의 답을 채우기 위한 해설서이다. 학생은 자신의 프로젝트를 네 직무 언어 중 하나로 다시 번역해야 한다.

앞의 70 페이지가 GitHub 포트폴리오를 점검하고 고치는 실습 흐름이라면, 심화 파트는 그 결과를 직무별 이야기로 바꾸는 단계이다. 같은 프로젝트라도 백엔드 지원자는 구조와 안정성을, 프론트엔드 지원자는 사용자의 판단 흐름을, 데이터 지원자는 수집·정제·검증 구조를, 보안 지원자는 위협과 통제의 근거를 앞에 세워야 한다.

중요한 것은 모든 직무를 다 잘 보이려는 욕심을 줄이는 것이다. 본인의 지원 직무를 하나 정한 뒤, 나머지 직무 요소는 보조 근거로 배치한다. 예를 들어 데이터 지원자는 보안 로그를 '감사 데이터 설계'의 근거로 사용할 수 있고, 보안 지원자는 같은 로그를 '공격 탐지와 사고 추적'의 근거로 사용할 수 있다.

학생이 읽어야 할 핵심

- README 상단의 한 줄 소개를 지원 직무 언어로 다시 작성한다.
- 대표 링크 3 개 중 최소 1 개는 직무별 증거 페이지로 연결한다.
- 면접 답변은 기능 나열이 아니라 문제, 판단, 증거, 결과 순서로 말한다.

README 에 녹이는 방식

직무	앞에 세울 질문	대표 증거
Backend	이 시스템은 안정적으로 동작하는가?	아키텍처, 주문 흐름, 리스크 검증, 감사 로그
Frontend	사용자가 실수 없이 판단할 수 있는가?	화면 흐름, 상태 표시, 차트, 운영 UX
Data	나중에 분석 가능한 데이터인가?	데이터 모델, 품질 기준, 집계 전략, 성과 지표
Security	무엇을 보호했고 어떻게 검증했는가?	위협 모델, 인증 방어, 암호화, 감사 로그
면접에서 꺼낼 문장 “제 프로젝트는 기능 구현만 보여주는 저장소가 아니라, 지원 직무 기준으로 어떤 문제를 어떻게 판단하고 검증했는지 보여주는 포트폴리오입니다.”		

Backend 심화: 단순 CRUD 를 넘어 운영 구조로 설명하기

이 페이지의 결론
백엔드 포트폴리오는 엔드포인트 목록보다 책임 분리, 데이터 일관성, 장애 대응, 감사 가능성을 보여줄 때 강해집니다.

백엔드 전공자 문서의 핵심은 ‘주문, 리스크, 감사 로그, API 호출 제한, 장애 대응까지 고려한 구조’이다. 학생 프로젝트에서도 이 관점을 가져오면 단순 계산기나 쇼핑물 프로젝트가 훨씬 성숙해진다. 예를 들어 주문 기능을 만들었다면 주문 요청, 재고 검증, 결제 결과, 실패 로그, 재시도 정책을 하나의 흐름으로 설명할 수 있어야 한다.

Java Core 와 Python Brain 을 gRPC 로 분리한 사례는 좋은 백엔드 설명이다. 핵심은 기술 이름이 아니라 책임 경계이다. 안정성이 중요한 주문과 계좌 처리는 Java 가 맡고, 실험과 추론이 많은 AI 판단은 Python 이 맡는 식으로 장애 전파 범위와 변경 속도를 분리했다는 점을 설명해야 한다.

학생이 읽어야 할 핵심

- Service Layer, Risk Service, Order Service, API Client, Audit Service 처럼 책임이 분명한 이름을 사용한다.
- JPA/MyBatis 선택은 취향이 아니라 운영 가시성, SQL 통제, 도메인 복잡도 관점에서 설명한다.
- 외부 API 장애, 재시도, 호출 제한, 자동 중지 같은 운영 시나리오를 README 에 기록한다.

README 에 녹이는 방식

아쉬운 표현	백엔드 직무 언어로 바꾼 표현
주문 API 를 만들었다.	주문 전 리스크 검증, 주문 요청 감사, 외부 API 호출, 결과 저장을 하나의 거래 흐름으로 설계했다.
MyBatis 를 사용했다.	금융 도메인의 SQL 추적성과 운영 대응을 위해 MyBatis 로 실제 실행 쿼리를 명확히 관리했다.
AI 기능을 연결했다.	AI 추론은 Python 서버로 분리하고, 주문 안정성이 필요한 영역은 Java 백엔드에서 책임지도록 설계했다.
장애 처리했다.	외부 API 실패 시 재시도 후 오류 로그와 사용자 알림을 남기고, 자동매매 중지를 선택할 수 있게 했다.
면접에서 꺼낼 문장 “단순히 API 를 호출한 것이 아니라, 주문 전후의 리스크와 감사 흐름까지 백엔드 책임으로 설계했습니다.”	

Backend README 구성 예시

이 페이지의 결론

백엔드 README 는 아키텍처, 주요 흐름, 설계 선택, 운영 고려사항이 차례로 보여야 한다.

백엔드 지원자의 README 는 기능 소개보다 시스템이 어떻게 안전하게 동작하는지 보여줘야 한다. 전체 아키텍처에서는 사용자, Controller, Service, Risk, Order, API Client, DB, Redis, Mail, KMS 같은 구성요소가 어떤 책임을 갖는지 설명한다. 그다음 주문 처리 흐름이나 인증 흐름을 시퀀스로 보여주면 면접관이 질문하기 쉬워진다.

학생들이 자주 놓치는 부분은 운영 환경이다. 로컬에서 돌아가는 앱은 많지만, 배포 가능한 백엔드처럼 보이려면 환경 변수, secret 외부화, SMTP, Redis, 장애 로그, 관리자 계정 절차 같은 운영 요소가 최소한 문서에 있어야 한다.

학생이 읽어야 할 핵심

- README 상단에 '무엇을 안정적으로 처리하는 백엔드인가'를 한 문장으로 작성한다.
- 핵심 흐름은 Mermaid sequenceDiagram 또는 단계표로 설명한다.
- 대표 설계 선택 3 개를 골라 선택 이유와 포기한 점을 적다.

README 에 녹이는 방식

섹션	포함할 내용
한 줄 소개	도메인, 핵심 책임, 기술 스택, 운영 수준
아키텍처	Controller-Service-DB-외부 API-캐시-메일-secret 흐름
핵심 흐름	주문, 인증, 결제, 알림, 장애 대응 중 대표 1~2 개
운영 고려	로그, 감사, 재시도, 호출 제한, 관리자 절차
면접 포인트	책임 분리, 트랜잭션, 데이터 일관성, 장애 대응
면접에서 꺼낼 문장 “저는 백엔드를 단순 기능 묶음이 아니라, 실패했을 때 추적하고 멈출 수 있는 운영 구조로 설명하려 했습니다.”	

Frontend 심화: 예쁜 화면보다 안전한 판단 흐름

이 페이지의 결론

프론트엔드 포트폴리오는 화면을 만들었다는 말보다 사용자가 더 정확하고 안전하게 행동하도록 설계했다는 근거가 중요하다.

프론트엔드 문서의 핵심은 '주식 투자라는 고위험 도메인에서 사용자가 실수 없이 판단하고 주문할 수 있도록 화면 흐름을 설계했다'는 점이다. 이 관점은 쇼핑물, 예약, 학사 시스템, 관리자 도구에도 그대로 적용된다. 사용자가 어떤 상황에서 어떤 정보를 보고 어떤 행동을 해야 하는지를 화면 흐름으로 설명해야 한다.

특히 증권사 코드값을 select box와 도움말로 바꾸고, 실주문·미수 가능·자동매매 같은 위험 행동을 상태 배지와 경고 문구로 드러낸 사례는 프론트엔드 포트폴리오에서 강력한다. 프론트엔드는 버튼과 CSS가 아니라 사용자의 실수를 줄이는 인터페이스라는 메시지를 준다.

학생이 읽어야 할 핵심

- 화면 목록보다 사용자 여정과 의사결정 흐름을 먼저 보여준다.
- 복잡한 코드값이나 도메인 용어를 사용자가 이해할 언어로 바꾼 사례를 적다.
- 차트, 상태 배지, 경고, 잠금, 관리자 화면은 모두 사용자 사고를 줄이는 장치로 설명한다.

README에 녹이는 방식

아쉬운 표현	프론트엔드 직무 언어로 바꾼 표현
화면을 만들었다.	사용자가 종목 검색, 주문 가능 금액 확인, 리스크 상태 확인을 한 흐름에서 처리하도록 화면을 설계했다.
차트를 넣었다.	현재가, 이동평균선, 주문 마커, 보유 수익률을 한 화면에 배치해 판단 근거를 줄였다.
관리자 페이지 구현	회원 승인, 권한 회수, 운영 로그 확인을 분리해 운영자가 사고를 빠르게 감지하도록 했다.
Bootstrap 사용	반복 사용자가 빠르게 스캔할 수 있도록 정보 밀도와 정렬을 우선했다.
면접에서 꺼낼 문장 “저는 화면을 예쁘게 만드는 것보다 사용자가 실수하지 않도록 돕는 프론트엔드를 중요하게 생각했습니다.”	

Frontend README 구성 예시

이 페이지의 결론

프론트엔드 README는 화면 캡처 모음이 아니라 사용자의 문제와 화면 설계 판단을 연결해야 한다.

프론트엔드 지원자는 README 상단에 대표 화면 이미지를 둘 수 있지만, 이미지만 있으면 부족하다. 그 화면이 어떤 사용자 문제를 해결하는지, 어떤 상태를 보여주는지, 어떤 실수를 줄이는지 설명해야 한다. 검색 결과가 1 건이면 자동 선택하고 여러 건이면 목록을 보여주는 작은 설계도 사용자 흐름을 깊이 생각했다는 근거가 된다.

운영자 화면도 좋은 포트폴리오 소재이다. 관리자 화면은 사용자 승인, 권한 회수, 오류 로그, API 상태처럼 반복 운영 업무를 줄이는 방향으로 설명하면 단순 CRUD 보다 높은 수준으로 보인다.

학생이 읽어야 할 핵심

- README에 대표 화면 2~3 개만 넣고 각 화면 아래에 '이 화면이 줄인 문제'를 작성한다.
- 사용자 흐름도는 로그인, 검색, 확인, 주문, 결과 확인처럼 행동 순서로 그립니다.
- 접근성, 반응형, 상태관리, 성능은 실제 사용자 문제와 연결될 때 설득력이 생깁니다.

README에 녹이는 방식

섹션	포함할 내용
한 줄 소개	도메인, 사용자 문제, 화면 설계 목표
사용자 흐름	로그인, 검색, 선택, 확인, 실행, 결과 확인
대표 화면	상태 배지, 경고, 차트, 입력 검증, 관리자 화면
설계 판단	왜 이 흐름이 사용자의 실수를 줄이는지
면접 포인트	UX 판단, 상태관리, 접근성, 운영 화면
면접에서 꺼낼 문장 “프론트엔드에서도 보안과 리스크 상태를 명확히 보여주는 것이 중요하다고 판단했습니다.”	

Data 심화: 저장보다 분석 가능한 구조

이 페이지의 결론

데이터 포트폴리오는 데이터를 많이 모았다는 말보다 나중에 어떤 질문에 답할 수 있게 설계했는지가 중요하다.

데이터 전공자 문서의 핵심은 시장 데이터, 주문 체결 데이터, AI 판단 로그를 재학습과 성과 분석에 활용할 수 있도록 수집·정제·집계·검증 구조를 설계했다는 점이다. 학생 프로젝트에서도 '데이터를 받았다'에서 멈추지 말고, 어떤 질문에 답하기 위해 어떤 단위로 저장했는지 설명해야 한다.

Tick 원본 전체 저장 대신 캔들 집계 방식을 선택한 사례는 데이터 설계 판단을 보여준다. 모든 데이터를 저장하는 것이 항상 좋은 선택은 아니다. 비용, 조회 성능, 분석 목적을 함께 고려해 어떤 데이터를 어떤 형태로 남길지 결정했다는 점이 중요하다.

학생이 읽어야 할 핵심

- Business Question 이 먼저 있고, 그 질문에 필요한 데이터 단위를 설계한다.
- 원본 저장, 집계 저장, 로그 저장 중 무엇을 선택했는지 이유를 작성한다.
- AI 판단 로그, 인증 실패 로그, 주문 감사 로그처럼 나중에 분석할 수 있는 이벤트 구조를 설명한다.

README 에 녹이는 방식

아쉬운 표현	데이터 직무 언어로 바꾼 표현
데이터를 저장했다.	AI 판단 결과와 실제 결과를 비교할 수 있도록 입력 해시, 판단, 확신도, 실제 가격을 함께 저장했다.
차트 데이터를 만들었다.	Raw tick 전체 저장 대신 OHLCV 캔들 집계로 비용과 조회 성능을 균형 있게 설계했다.
수익률 계산	실현손익, 평가손익, 총손익을 분리해 성과 분석의 기준을 명확히 했다.
로그 저장	로그인 실패와 API 장애 데이터를 분리해 보안 이벤트와 운영 장애를 각각 분석할 수 있게 했다.
면접에서 꺼낼 문장 “데이터를 많이 저장하는 것보다 나중에 분석 가능한 형태로 저장하는 것이 중요하다고 판단했습니다.”	

Data README 구성 예시

이 페이지의 결론

데이터 README 는 데이터 출처, 처리 기준, 지표 정의, 검증 가능성을 보여주는 신뢰 문서이다.

데이터 지원자의 README 는 분석 결과보다 데이터 구조와 품질 관리가 먼저 보이면 좋다. 한글 깨짐을 막기 위한 인코딩 기준, 사용자별 데이터 분리, 민감 정보 미저장, 마스킹 원본 보관 여부 같은 세부 기준은 실무 감각을 보여준다.

또한 수익률, 승률, MDD, 확신도, 지연시간 같은 지표는 반드시 정의가 필요하다. 지표 정의가 없으면 결과가 좋아 보여도 신뢰하기 어렵다. 데이터 포트폴리오는 결과표보다 그 결과가 어떻게 계산되었는지 설명할 때 수준이 올라간다.

학생이 읽어야 할 핵심

- README 에 데이터 아키텍처, 주요 테이블, 지표 정의, 품질 기준을 분리해 둔다.
- 결측치, 이상치, 중복, 민감 정보 처리 기준을 짧게라도 작성한다.
- 분석 결과에는 한계와 다음 검증 계획을 함께 붙인다.

README 에 녹이는 방식

섹션	포함할 내용
Business Question	이 데이터가 답해야 하는 의사결정 질문
Data Overview	출처, 기간, 규모, 주요 컬럼, 라이선스
Data Model	테이블 관계, 이벤트 로그, 집계 단위
Quality Rule	결측, 이상치, 인코딩, 마스킹, 민감값 제외
Insight	지표 정의, 결과 해석, 한계, 다음 검증
면접에서 꺼낼 문장 “AI 판단 로그는 모델의 책임성을 확인할 수 있도록 입력 해시, 판단, 확신도, 실제 결과를 함께 남겼습니다.”	

Security 심화: 보안을 마지막에 붙이지 않기

이 페이지의 결론

보안 포트폴리오는 보안 기능을 나열하는 문서가 아니라, 시스템 전반에 어떤 위협을 가정하고 어떤 통제를 넣었는지 보여주는 문서이다.

보안 전공자 문서의 핵심은 API 키 암호화, 2 차 인증, 개인정보 마스킹, 주문 감사, 장애 시 자동매매 중지까지 보안 중심 구조를 설계했다는 점이다. 학생 프로젝트에서도 보안은 기능 마지막에 붙이는 체크박스가 아니라 인증, 설정, 주문, 로그, 운영 전반에 들어가야 한다.

Redis 기반 인증 시도 제한과 로그인 실패 로그 저장은 좋은 예이다. 단순히 로그인 기능이 있다는 말보다, 실패 횟수를 제한하고 공격 분석에 필요한 이벤트 유형, 이메일, IP, User-Agent, 실패 사유만 저장했다는 설명이 훨씬 강하다. 동시에 비밀번호와 TOTP 코드는 저장하지 않았다는 말이 개인정보 보호 감각을 보여준다.

학생이 읽어야 할 핵심

- 보호 대상, 위협, 통제, 검증 증거를 한 세트로 설명한다.
- 암호화, 마스킹, 2 차 인증, 실패 제한, 감사 로그는 서로 연결된 흐름으로 보여준다.
- 보안 캡처나 로그는 반드시 민감 정보를 제거하고 교육·본인 프로젝트 범위 안에서만 제시한다.

README 에 녹이는 방식

아쉬운 표현	보안 직무 언어로 바꾼 표현
2 차 인증 구현	TOTP 기반 2 차 인증을 적용하고 실패 시도는 Redis 와 감사 로그로 추적했다.
API 키 저장	한국투자 API key 와 secret 을 AES-GCM 으로 암호화 저장하고 화면 응답에서는 마스킹했다.
로그 남김	주문 요청과 결과를 감사 로그로 남겨 사고 발생 시 누가 언제 어떤 조건으로 요청했는지 추적할 수 있게 했다.
DB 비밀번호 관리	개발/운영 DB 비밀번호는 AWS KMS 암호문으로 외부화해 설정 파일 평문 노출을 줄였다.
면접에서 꺼낼 문장 “보안을 기능 마지막에 붙인 것이 아니라 인증, 설정, 주문, 로그, 운영 전반에 적용하려고 했습니다.”	

Security README 구성 예시

이 페이지의 결론

보안 README는 공격 방법을 과시하는 문서가 아니라, 보호 대상과 검증 가능한 방어 흐름을 설명하는 문서이다.

보안 지원자는 README에 Threat Model을 반드시 넣는 것이 좋다. 보호 대상은 API 키, 계좌번호, TOTP Secret, 비밀번호, DB 비밀번호처럼 구체적이어야 한다. 위협은 계정 탈취, 민감정보 유출, 주문 사고, 운영 실수처럼 영향 중심으로 정리한다.

보안 문서에서 중요한 것은 책임 있는 공개이다. 취약점 재현을 지나치게 자세히 쓰기보다, 어떤 위협을 발견했고 어떤 통제를 적용했으며 어떤 로그나 테스트로 방어를 확인했는지 보여주는 편이 안전하고 전문적이다.

학생이 읽어야 할 핵심

- README 목차를 Executive Summary, Threat Model, Evidence, Mitigation, Lesson Learned 순서로 둡니다.
- 보호 대상 표에는 저장 방식과 화면/API 응답 방식을 함께 적다.
- 장애 대응, 호출 제한, 자동 정지는 보안과 운영 안정성을 함께 보여주는 좋은 소재이다.

README에 녹이는 방식

섹션	포함할 내용
Executive Summary	무엇을 보호했고 어떤 통제를 설계했는가
Threat Model	계정 탈취, 민감정보 유출, 주문 사고, 운영 실수
Protection	암호화, 마스킹, 2FA, rate limit, secret 외부화
Evidence	실패 로그, 차단 응답, 감사 로그, 설정 캡처
Lesson Learned	남은 위험과 다음 개선 계획

면접에서 꺼낼 문장

“보안 컴플라이언스에 따라 비밀번호 및 TOTP(Time-based One-Time Password) 코드는 로그 저장 단계에서 전면 배제했습니다. 시스템 내 침입 징후 및 패턴 분석에 필수적인 포렌식 데이터만을 선별하여 저장하는 구조를 구축했습니다..”

직무별 내용을 하나의 포트폴리오로 묶기

이 페이지의 결론

좋은 GitHub 포트폴리오는 직무별 강점을 따로 보여주되, 결국 하나의 프로젝트 이야기로 연결된다.

백엔드, 프론트엔드, 데이터, 보안은 서로 분리된 섬이 아니다. 같은 투자 시스템에서도 백엔드는 주문 안정성을, 프론트엔드는 사용자 오조작 방식을, 데이터는 판단과 감사의 근거를, 보안은 민감정보 보호와 사고 추적성을 담당한다. 학생 프로젝트도 이처럼 역할별 관점을 연결하면 훨씬 입체적으로 보인다.

최종 README에서는 지원 직무를 가장 앞에 세우고 나머지 직무 요소를 보조 근거로 배치한다. 백엔드 지원자라면 보안은 운영 안정성의 일부로, 데이터는 감사 로그와 성능 분석의 일부로 설명한다. 데이터 지원자라면 백엔드 구조는 데이터가 생성되는 맥락으로, 보안 로그는 분석 가능한 이벤트 데이터로 설명한다.

학생이 읽어야 할 핵심

- 첫 화면은 지원 직무 중심으로 구성하되, 다른 직무 요소는 증거 링크로 연결한다.
- 대표 링크 3 개는 구조, 증거, 면접 답변으로 역할을 나눕니다.
- 문서가 길어질수록 목차와 링크 설명이 중요하다.

README 에 녹이는 방식

대표 링크	역할	예시
1 번 링크	핵심 직무 증거	Backend 아키텍처 또는 Data Overview
2 번 링크	검증 증거	주문 감사 로그, 차트, 테스트 결과, 보안 차단 로그
3 번 링크	면접 답변 증거	STAR 사례, PR, Issue, Trade-off 표
보조 링크	관심 확장	Frontend 화면 흐름, Security Threat Model, Data Model

면접에서 꺼낼 문장

“이 프로젝트에서 제 중심 역할은 ○○이지만, 운영 가능한 제품으로 보이도록 데이터와 보안 근거까지 함께 연결했습니다.”